
Software Engineering

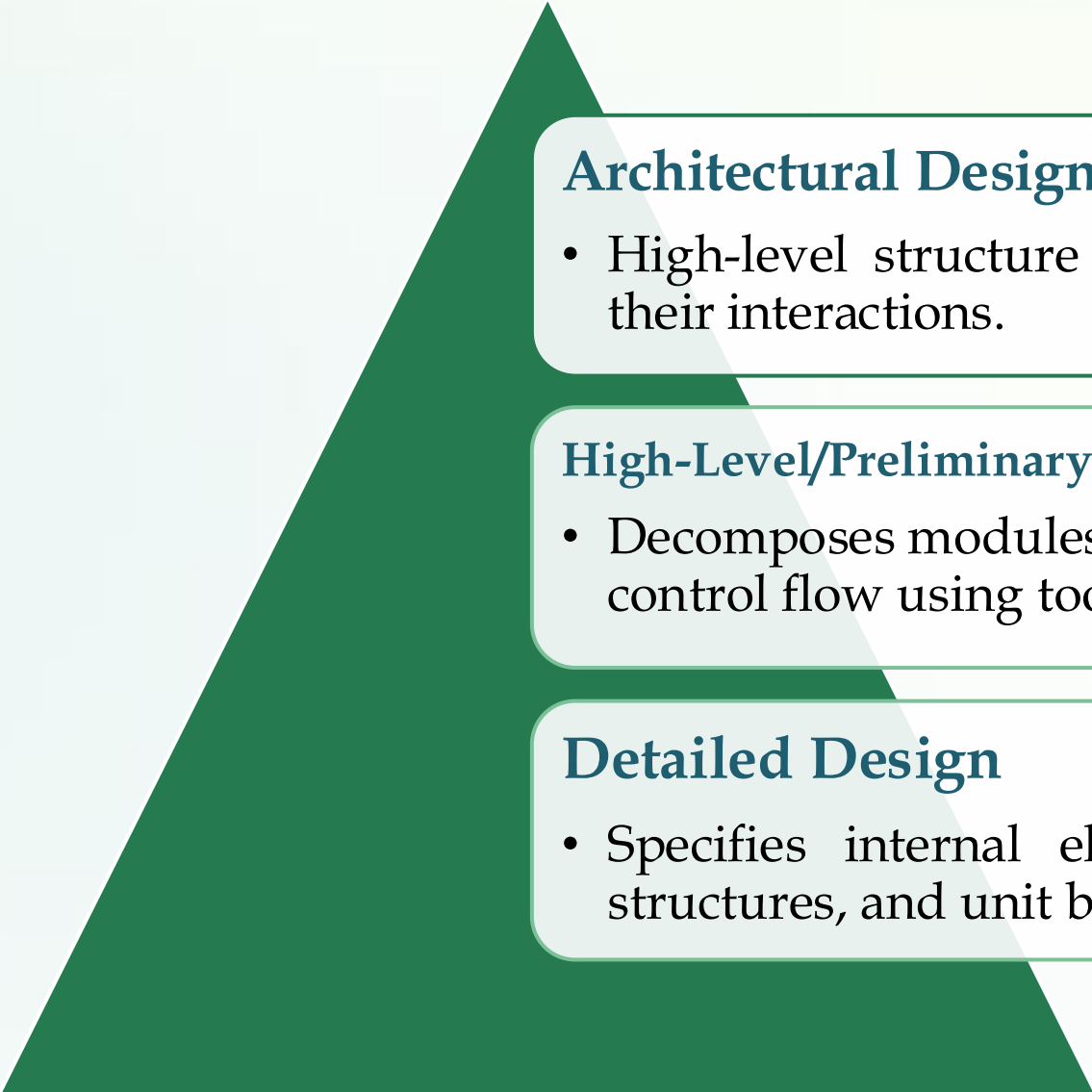
BCSE301L

Module 4 : Software Design

Software Design

- ❖ The process of defining the architecture, components, interfaces, and data for a system to satisfy specified requirements.
- ❖ It bridges the gap between problem definition and solution implementation, creating a blueprint that guides coding and development.
- ❖ It transforms the **Software Requirements Specification** (SRS) into detailed plans, including modules, data structures, and algorithms.

Design Levels



Architectural Design

- High-level structure identifying major components and their interactions.

High-Level/Preliminary Design

- Decomposes modules, defines interfaces, and outlines control flow using tools like structure charts or UML.

Detailed Design

- Specifies internal elements such as algorithms, data structures, and unit behaviours.

Design Process (In simple Terms)

Data design

This phase produces the data structures.



Architectural design

This phase produces the structural units (classes).



Interface design

This phase specifies the interfaces between the units.



Procedural design

This phase specifies the algorithms of each method.

Software Design Concepts

❖ These provide foundational principles for creating structured, maintainable systems.

Abstraction

Modularity

Architecture

Information
hiding

Separation of
Concerns

Functional
independence

Refinement

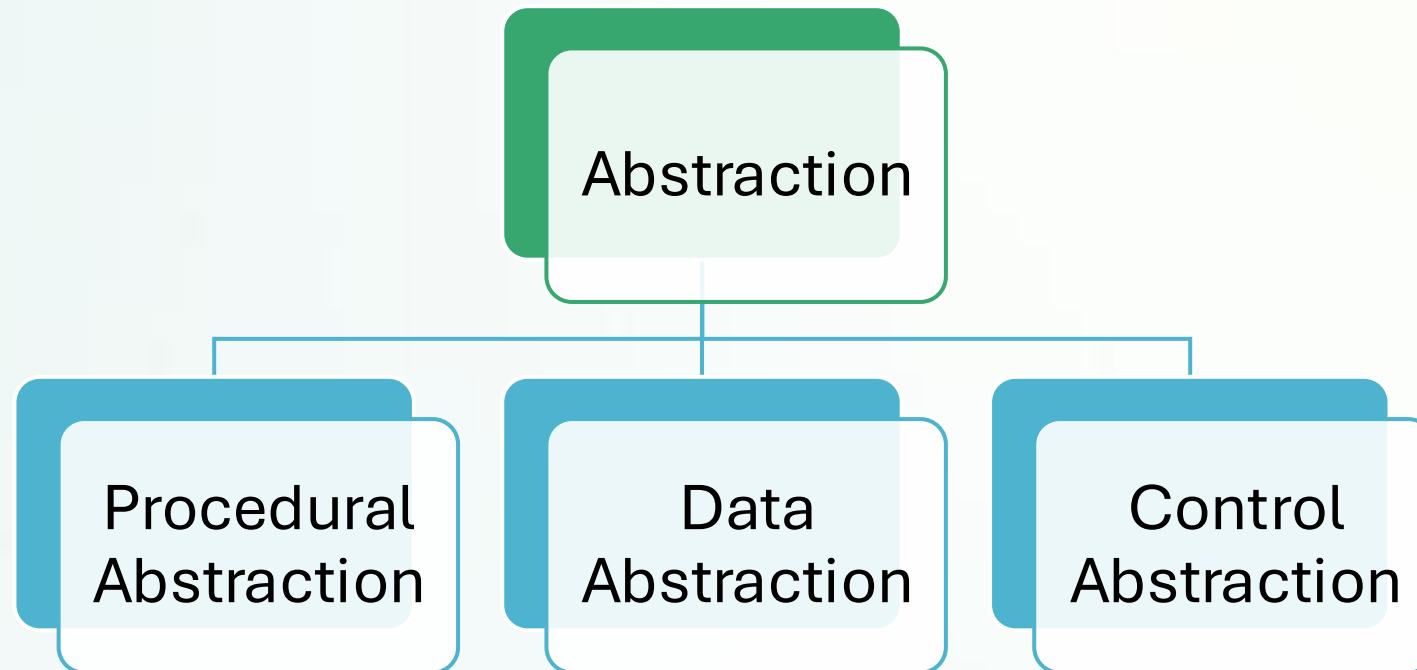
Design patterns

Coupling

Cohesion

Software Design Concepts – 1. Abstraction

- ❖ Abstraction simplifies complex systems by **focusing on essential features** while **hiding irrelevant details**.
- ❖ It uses mechanisms like **functional (procedural steps)**, **data (structures)**, or **control (flow) abstraction** to manage complexity.



1. Abstraction

Data Abstraction

- A named collection of data that describes a data object. At the **highest level**, we focus on the "**what**" (e.g., a User), while at **lower levels**, we **define the specific data structures** (e.g., String name, int ID).

Procedural Abstraction

- A sequence of instructions that have a specific and limited function. The **name of the procedure** implies its function, but the **internal details are hidden** (e.g., CalculatePayroll).

Control Abstraction

- Implies a program control mechanism without specifying internal details. This often involves how the **system handles events or synchronization**.

1. Abstraction - Example

❖ Data Abstraction

- ❖ When a designer defines a class like Account, they are using data abstraction. They don't need to worry about how the double or String is stored in memory bits; they only care about the high-level representation of the account details.

❖ Procedural Abstraction

- ❖ When you call the method `withdraw()`, you are invoking a procedural abstraction. The caller knows that money will be removed from the account, but they do not need to know the specific logic (the "how") of the subtraction or the database updates happening inside that method.

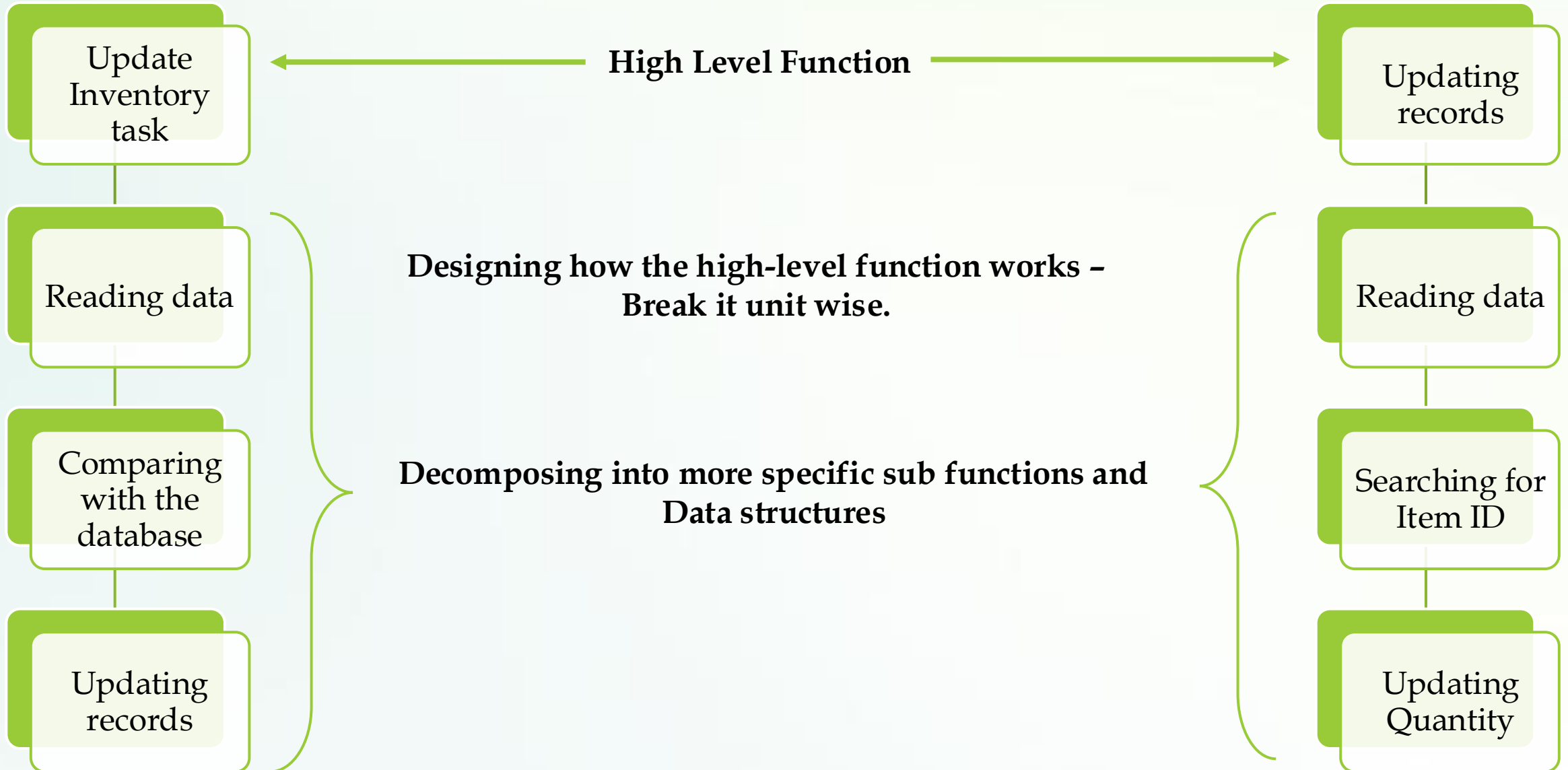
❖ Control Abstraction

- ❖ This is often seen in how a system handles loops or branching at a high level. In a design context, it refers to hiding the complexity of the control flow—for example, a `syncWithBankServer()` method hides the complex retry logic, timeouts, and network handshakes required to maintain control of the data flow

2. Refinement

- ❖ Refinement is presented as a **top-down software design strategy** that complements abstraction.
- ❖ Beginning with a high-level function, the designer progressively adds detail through successive steps, decomposing the initial statement into increasingly specific sub-functions and data structures, a process known as stepwise refinement.

2. Refinement



3. Modularity

- ❖ if we subdivide software indefinitely, the effort required to develop it decreases;
- ❖ However, as the number of modules grows, the effort required to integrate those modules also grows.
- ❖ The goal of **modularity** is to find the "sweet spot" where total cost is minimized.



Integration of Sub Modules increases efforts and Cost

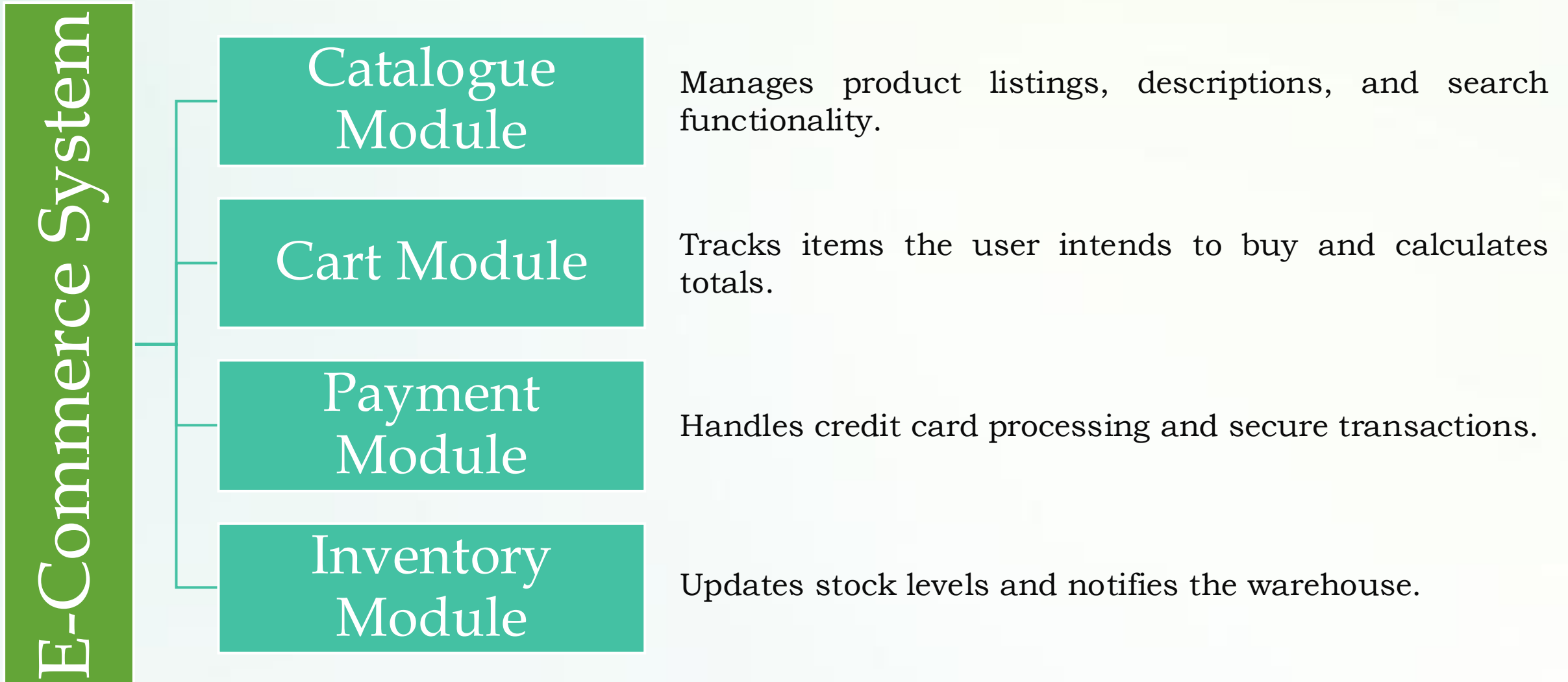


Dividing Software Product into simple n number of sub-Modules decreases the developing Effort

3. Modularity

- ❖ It is based on the principle of **Separation of Concerns**.
- ❖ Instead of one monolithic block of code, the system is broken into components (modules) that can be integrated to satisfy the problem requirements.

3. Modularity - Example



3. Modularity - Benefits

❖ Manageability

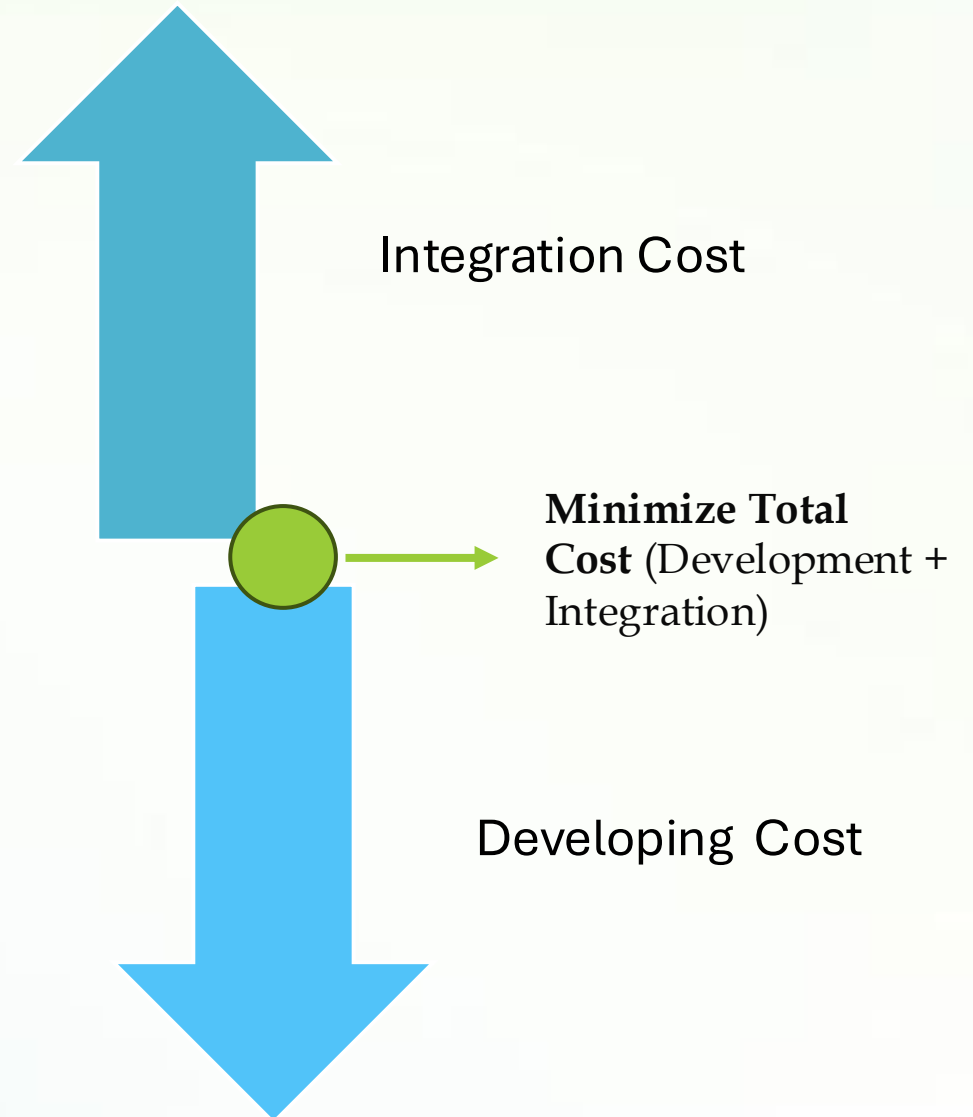
- ❖ It is easier for a human to understand a 100-line module than a 10,000-line program.

❖ Parallel Development

- ❖ Different teams can work on the PaymentModule and the CatalogModule simultaneously without interfering with each other.

❖ Maintenance

- ❖ If the tax laws change, you only need to modify the logic inside the CartModule (specifically the tax calculation part) rather than searching through the entire codebase.



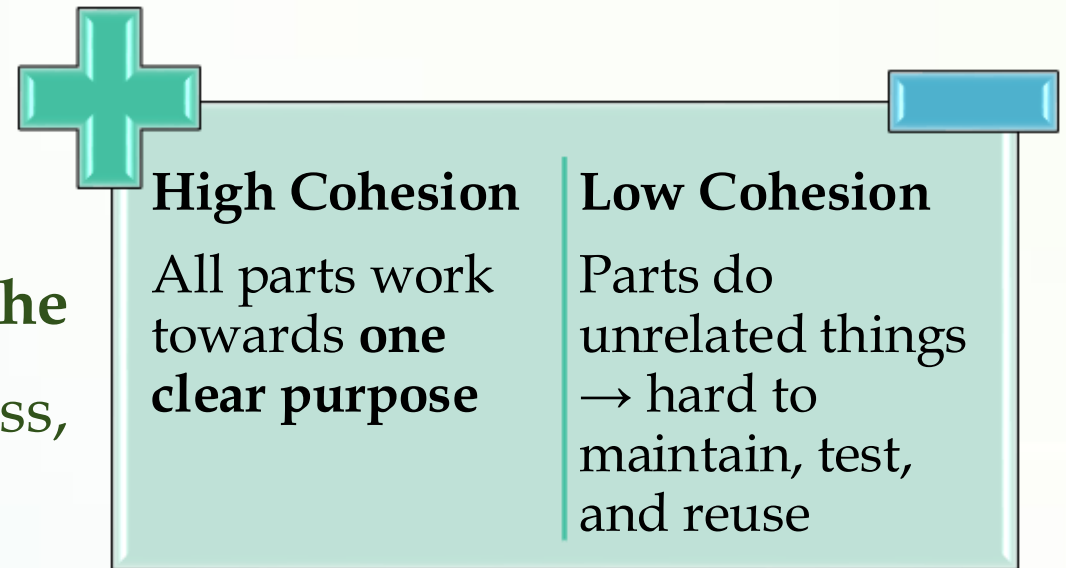
4. Functional Independence

- ❖ Functional Independence is the **direct outgrowth of modularity, abstraction, and information hiding.**
- ❖ It is achieved by developing modules with a **"single-minded"** function and an **"aversion"** to **excessive interaction** with other modules.
- ❖ **Independence** is assessed using two qualitative criteria:
 - ❖ Cohesion
 - ❖ Coupling.

4. Functional Independence - Cohesion

❖ Cohesion is a measure of **how focused a single module, class, or function is**. Think of it as the "single responsibility" principle in action: *a highly cohesive module does one thing and does it well*

❖ Cohesion measures **how closely the responsibilities inside a single module** (class, function, or file) **are related to each other**.



4. Functional Independence - Cohesion

❖ 1. Coincidental Cohesion

- ❖ Elements grouped randomly with no logical relationship.

❖ Problem

- ❖ Changing tax logic might accidentally break email functionality because they share the same class.

BAD: Random utilities thrown together

```
class MiscUtils:
```

```
    def calculate_tax(self, amount): # Finance  
        return amount * 0.15
```

```
    def send_email(self, to, body): # Communication  
        print(f"Emailing {to}")
```

```
    def resize_image(self, path): # Graphics  
        print(f"Resizing {path}")
```

4. Functional Independence - Cohesion

❖ 2. Logical Cohesion

- ❖ Elements grouped by category/type, not by purpose

// BAD: "All handlers" grouped together

```
class InputHandler {  
    handleKeyboard(event) { /* ... */ } // Different purpose  
    handleMouse(event) { /* ... */ } // Different purpose  
    handleVoice(command) { /* ... */ } // Different purpose  
}
```

❖ Problem

- ❖ These handle different input types with different logic—better as separate classes

4. Functional Independence - Cohesion

❖ 3. Temporal Cohesion

- ❖ Elements grouped because they run at the same time (e.g., startup/shutdown)

❖ Problem

- ❖ Adding a new startup task forces changes to unrelated logic. Hard to test individually.

BAD: Things that happen "at startup" but aren't related

```
def on_app_start():  
    connect_to_database() # Infrastructure  
    show_welcome_popup() # UI  
    log_user_activity()   # Analytics  
    preload_images()      # Assets
```

4. Functional Independence - Cohesion

❖ 4. Procedural Cohesion

- ❖ Elements grouped because they execute in sequence—but not tightly related.

BAD: Sequential steps with different responsibilities
def process_order(order):
 validate_order(order) # Validation
 save_to_database(order) # Persistence
 send_confirmation_email() # Communication
 update_inventory(order) # Business logic

❖ Problem

- ❖ Mixing validation, DB, email, and inventory logic makes testing difficult and violates Single Responsibility Principle.

4. Functional Independence - Cohesion

❖ 5. Communicational Cohesion

❖ Elements operate on the same data

GOOD: All methods work with User data

```
class UserService:
```

```
    def create_user(self, user_data):
```

```
        # ...
```

```
    def update_user_email(self, user_id, new_email):
```

```
        # ...
```

```
    def delete_user(self, user_id):
```

```
        # ...
```

❖ Working

❖ Changes to user data structure affect all methods predictably. Easy to test and maintain.

4. Functional Independence - Cohesion

❖ 6. Sequential Cohesion

- ❖ Output of one element becomes input to the next (pipeline).

```
# GOOD: Data flows cleanly through stages
def generate_report(data):
    cleaned = clean_data(data)    # Output → Input
    analyzed = analyze(cleaned)   # Output →
    Input
    formatted = format_report(analyzed) # Output
    → Input
    return formatted
```

❖ Working

- ❖ Each step has one job. Easy to insert logging/testing between stages

4. Functional Independence - Cohesion

❖ 7. Functional Cohesion

(Best) All parts contribute to a SINGLE task

❖ Working

- ❖ Easy to name (calculate_total_price says exactly what it does)
- ❖ Easy to test (one input → one output)
- ❖ Easy to reuse (no hidden side effects)
- ❖ Changes are localized

```
# EXCELLENT: One clear purpose
def calculate_total_price(cart_items):
    subtotal = sum(item.price * item.qty for item in cart_items)
    tax = subtotal * 0.10
    shipping = 5.00 if subtotal < 50 else 0
    return subtotal + tax + shipping
```

Another example:

```
class PasswordValidator:
```

```
    def is_valid(self, password):
        return (self._has_min_length(password) and
                self._has_uppercase(password) and
                self._has_number(password))
```

```
    def _has_min_length(self, pwd): ... # All support ONE goal:
validation
    def _has_uppercase(self, pwd): ...
    def _has_number(self, pwd): ...
```

4. Functional Independence - Cohesion

❖ Identify the Cohesion

```
class UserManager:  
    def create_user(self, data): ...  
    def send_welcome_email(self, user): ...  
    def generate_pdf_report(self, user): ...  
    def backup_database(self): ...  
    def log_activity(self, msg): ...
```

❖ if it is not best cohesion, convert it to so that it has best cohesion

4. Functional Independence - Cohesion

❖ Converted Code - Identify the Cohesion

Each class has ONE responsibility

```
class UserService:
```

```
    def create_user(self, data): ...
```

```
class EmailService:
```

```
    def send_welcome_email(self, user): ...
```

```
class ReportService:
```

```
    def generate_pdf_report(self, user): ...
```

```
class BackupService:
```

```
    def backup_database(self): ...
```

```
class Logger:
```

```
    def log_activity(self, msg): ...
```

4. Functional Independence - Cohesion

Benefit	Low Cohesion	High Cohesion
Testing	Hard (many side effects)	Easy (isolated units)
Reusability	Rarely reusable	Plug-and-play modules
Maintenance	Change one thing → breaks others	Changes stay localized
Readability	"What does this class actually do?"	Clear purpose from name alone
Teamwork	Merge conflicts everywhere	Parallel development

4. Functional Independence - Coupling

❖ Coupling measures how dependent one module (class, function, service) is on another. It answers:

If I change Module A, how much will Module B break?

Loose coupling

- Modules interact through simple, stable interfaces → changes stay localized

Tight coupling

- Modules are deeply entangled → change one, break many

4. Functional Independence - Coupling

❖ 1. Content Coupling (Worst)

❖ One module directly modifies or relies on the internal workings of another.

❖ **Problem:** If OrderProcessor changes `_items` to a dictionary tomorrow, `hack_order_processor` breaks silently.

BAD: Module B reaches into Module A's internals

```
class OrderProcessor:
```

```
    def __init__(self):  
        self._items = [] # Private internal state
```

```
    def add_item(self, item):  
        self._items.append(item)
```

Another module directly manipulates private state!

```
def hack_order_processor(processor):
```

```
    processor._items.clear() # Reaching into  
internals!  
    processor._items.append({"name": "free_item",  
"price": 0})
```

4. Functional Independence - Coupling

❖ 2. Common Coupling

❖ Multiple modules share global state (global variables, singletons).

❖ **Problem:** Impossible to track who changed the state → debugging nightmare.

// BAD: Shared global state

let currentUser = null; // Global variable

```
class LoginService {
```

```
  login(user) {
```

```
    currentUser = user; // Writes to global
```

```
  }
```

```
}
```

```
class Dashboard {
```

```
  show() {
```

```
    if (currentUser) { // Reads from global
```

```
      console.log(`Welcome ${currentUser.name}`);
```

```
    }
```

```
  }
```

```
}
```

// Disaster scenario:

```
new LoginService().login({name: "Alice"});
```

// ...later, somewhere else...

```
currentUser = null; // ❌ Any code can corrupt shared state!
```

```
new Dashboard().show(); // Broken!
```

4. Functional Independence - Coupling

❖ 3. External Coupling

❖ Modules depend on external formats/protocols (file formats, APIs, DB schemas).

❖ **Problem:** If the CSV provider adds a column or changes delimiters, your code breaks..

BAD: Hard-coded dependency on external format

```
def parse_csv_report(file_path):  
    with open(file_path) as f:  
        lines = f.readlines()  
        # Assumes EXACT format:  
        "id,name,price,date"  
        parts = lines[1].split(',') # Breaks if CSV  
        format changes  
        return {  
            'id': parts[0],  
            'name': parts[1],  
            'price': float(parts[2])  
        }
```

4. Functional Independence - Coupling

❖ 4. Control Coupling

❖ A module passes flags/commands that control how another module behaves.

❖ **Problem:** The function's behavior isn't predictable from its name alone. Hard to test all flag combinations.

BAD: Caller dictates internal behavior via flags

```
def process_order(order, is_test_mode=False,
skip_validation=False):
    if not skip_validation: # Caller controls internal
logic
        validate(order)

    if not is_test_mode:
        charge_payment(order)

    ship_order(order)
```

Usage:

```
process_order(order, is_test_mode=True,
skip_validation=True)
```

4. Functional Independence - Coupling

❖ 5. Stamp Coupling

❖ Passing entire objects / structures when only a small part is needed.

❖ Problem:

- ❖ Function signature lies about its needs ("I need a User" → really just needs email)
- ❖ Changing User structure might break this function even if .email is unchanged

BAD: Function receives whole User but only needs email

```
def send_welcome_email(user): #  
    # Receives entire User object  
    email_service.send(user.email,  
        "Welcome!") # Only uses .email
```

Usage:

```
send_welcome_email(current_user) #  
    # Passes name, address, password  
    # hash... unused!
```

4. Functional Independence - Coupling

❖ 6. Data Coupling

❖ Modules exchange minimal, primitive data needed for the task..

❖ Good Working:

❖ Easy to test (just pass numbers/strings)

❖ Changes to unrelated parts of the system won't break this

❖ Interface clearly communicates requirements

GOOD: Clear, minimal interface

```
class PaymentGateway:
```

```
    def charge(self, amount: float,  
currency: str): # Only needs 2  
primitives
```

```
    # Process payment...  
    return transaction_id
```

Usage:

```
gateway.charge(99.99, "USD") # No  
hidden dependencies
```

4. Functional Independence - Coupling

❖ 7. Message Coupling

- ❖ Modules communicate asynchronously via messages/events with no direct knowledge of each other.
- ❖ **Best Working:**
 - ❖ OrderService doesn't know EmailService exists
 - ❖ Can add new listeners (e.g., AnalyticsService) without touching OrderService
 - ❖ Services can fail independently (email down? orders still process)

4. Functional Independence - Coupling

EXCELLENT: Event-driven architecture

class OrderService:

```
def place_order(self, order):
```

```
    # ...process order...
```

```
    event_bus.publish("order_placed", {
```

```
        "order_id": order.id,
```

```
        "customer_email": order.email
```

```
    })
```

Completely decoupled consumers:

class EmailService:

```
def on_order_placed(event):
```

```
    send_email(event["customer_email"], "Your order is confirmed!")
```

class InventoryService:

```
def on_order_placed(event):
```

```
    reduce_stock(event["order_id"])
```

Register handlers (no direct references between services):

```
event_bus.subscribe("order_placed",  
    EmailService.on_order_placed)
```

```
event_bus.subscribe("order_placed",  
    InventoryService.on_order_placed)
```

4. Functional Independence - Coupling

Monolith with tangled dependencies

```
class ShoppingCart:
```

```
    def __init__(self):  
        self.items = []
```

```
    def checkout(self):
```

```
        # Directly creates DB connection (external coupling)
```

```
        db = DatabaseConnection("mysql://...")
```

```
        db.save(self.items)
```

Directly accesses global user (common coupling)

```
global current_user
```

```
    send_email(current_user.email, "Receipt") # Hard-  
    coded email logic
```

```
    # Directly manipulates payment internals (content  
    coupling)
```

```
    payment = PaymentProcessor()
```

```
    payment._balance -= self.total() # ❌ Reaching into  
    private state!
```

Identify the Coupling.

4. Functional Independence - Coupling

Clean separation of concerns

class ShoppingCart:

def __init__(self, items):

self.items = items

def checkout(self, payment_gateway, event_bus):

1. Only passes minimal data needed

transaction_id = payment_gateway.charge(self.total())

2. Publishes event instead of calling services directly

event_bus.publish("order_completed", {

"items": self.items,

"transaction_id": transaction_id

})

No knowledge of email, DB, or inventory systems!

Decoupled services:

class PaymentGateway:

def charge(self, amount): ... # Returns ID only

class EmailService:

def on_order_completed(event):

send_email(get_user_email(), "Receipt")

Composition at top level:

cart = ShoppingCart(items)

cart.checkout(PaymentGateway(), EventBus())

Identify the Coupling. It is now converted to?

4. Functional Independence - Coupling

High cohesion + Loose coupling = Maintainable software

	High Cohesion	Low Cohesion
Loose Coupling	Ideal: Focused modules that work independently	Tolerable: Unfocused but isolated modules
Tight Coupling	Fragile: Focused but brittle dependencies	Worst: Unfocused + entangled = maintenance nightmare

Classroom Activity

```
#include <iostream>

#include <fstream>

#include <string>

#include <vector>

#include <ctime>

#include <cstdlib> // For system() calls

using namespace std;

vector<double> history; // Shared history across ALL modules

string currentUser = "guest"; // Global user state

bool debugMode = false; // Global flag affecting behavior everywhere

ofstream logFile("calculator.log", ios::app); // Global file handle
```

Classroom Activity

```
class Calculator {
```

```
private:
```

```
    double currentValue;
```

```
    string displayBuffer;
```

```
    int errorCount;
```

```
    time_t lastOperationTime;
```

```
// Internal helper
```

```
void _updateHistory(double result) {
```

```
    history.push_back(result); // Modifies GLOBAL history directly!
```

```
}
```

Classroom Activity

public:

```
Calculator() {  
    currentValue = 0;  
    displayBuffer = "0";  
    errorCount = 0;  
    lastOperationTime = time(nullptr);  
  
    cout << "Calculator v1.0 starting...\n";  
    logfile << "Session started for user: " << currentUser << endl;  
    system("clear");  
    srand(time(0));  
}
```

Classroom Activity

```
void performOperation(string op, double operand, bool shouldLog = true, bool validateFirst = true) {
    if (validateFirst && operand == 0 && (op == "/" || op == "mod")) {
        cout << "Error: Division by zero!\n";
        errorCount++;
        if (shouldLog) logFile << "ERROR: Division by zero attempted\n";
    }
    return;
    double result = 0;
    if (op == "+") result = currentValue + operand;
    else if (op == "-") result = currentValue - operand;
    else if (op == "*") result = currentValue * operand;
    else if (op == "/") result = currentValue / operand;
    else if (op == "mod") result = fmod(currentValue, operand);
    else if (op == "pow") result = pow(currentValue, operand);
    else if (op == "sqrt") result = sqrt(operand); // Inconsistent: ignores currentValue!
    else {
        cout << "Unknown operation!\n";
        errorCount++;
        return;
    }
}
```

Classroom Activity

```
saveResultToFile(this, result);
currentValue = result;
    displayBuffer = to_string(result);
if (shouldLog) {
    logFile << currentUser << " performed " << op << " -> " << result << endl;
    lastOperationTime = time(nullptr); // Updates internal state based on
external decision
}

_updateHistory(result); // Breaks encapsulation via global access
}
```

Classroom Activity

```
void processUserInput(string input) {  
    // Step 1: Parse input (parsing logic)  
    size_t spacePos = input.find(' ');  
    if (spacePos == string::npos) {  
        cout << "Invalid format! Use: <op> <number>\n";  
        return;  
    }  
  
    string op = input.substr(0, spacePos);  
    double num = stod(input.substr(spacePos + 1));  
  
    // Step 2: Validate (validation logic mixed here)  
    if (op == "sqrt" && num < 0) {  
        cout << "Error: Cannot sqrt negative number!\n";  
        return;  
    }  
}
```

Classroom Activity

```
// Step 3: Execute operation (business logic)
```

```
performOperation(op, num, true, false); // Control Coupling via flags
```

```
// Step 4: Update UI (presentation logic)
```

```
cout << "Result: " << currentValue << "\n";
```

```
// Step 5: Check for easter egg (TOTALLY unrelated feature!)
```

```
if (currentValue == 42) {
```

```
    cout << "*** SECRET UNLOCKED: You found the answer to life! ***\n";
```

```
    system("echo '42' > /tmp/answer.txt"); // External Coupling: OS command
```

```
}
```

```
}
```

Classroom Activity

```
void showOutput(string formatType) {
    if (formatType == "screen") {
        cout << "Display: " << displayBuffer << endl;
    }
    else if (formatType == "hex") {
        cout << "Hex: 0x" << hex << (int)currentValue << dec << endl;
    }
    else if (formatType == "binary") {
        // Binary conversion logic crammed here
        int n = (int)currentValue;
        string bin = "";
        while (n > 0) { bin = to_string(n % 2) + bin; n /= 2; }
        cout << "Binary: " << (bin.empty() ? "0" : bin) << endl;
    }
    else if (formatType == "history") {
        cout << "History (" << currentUser << "):\n";
        for (double val : history) { // Uses GLOBAL history
            cout << " " << val << endl;
        }
    }
    else if (formatType == "stats") {
        // Analytics crammed into calculator class!
        cout << "Errors: " << errorCount << ", Ops: " << history.size() << endl;
    }
}
```

Classroom Activity

```
void cleanupAndExit() {  
    cout << "Saving session...\n";  
    logFile << "Session ended. Total ops: " << history.size() << endl;  
    logFile.close(); // Closes GLOBAL file handle  
  
    // OS-specific cleanup (External Coupling)  
    #ifdef _WIN32  
        system("pause");  
    #else  
        system("read -p 'Press Enter to exit...'");  
    #endif  
  
    // Memory "cleanup" (pointless in C++ but shows temporal grouping)  
    currentValue = 0;  
    displayBuffer = "";  
    history.clear(); // Modifies GLOBAL state!  
  
    cout << "Goodbye, " << currentUser << "!\n";  
    exit(0);  
}  
  
// Getter that exposes internal state (enables Content Coupling)  
double* getInternalValuePtr() {  
    return &currentValue; // DANGER: Allows external code to modify internals directly!  
}  
};
```

Classroom Activity

```
void saveResultToFile(Calculator* calc, double result) {  
    // Receives entire Calculator object but only uses 'result'  
    ofstream file("results.csv", ios::app);  
  
    // Assumes CSV format with specific columns  
    file << time(nullptr) << "," << currentUser << "," << result << endl;  
    file.close();  
}
```

Classroom Activity

```
void hackCalculatorValue(Calculator& calc, double newValue) {  
    double* ptr = calc.getInternalValuePtr(); // Gets pointer to private member  
    *ptr = newValue; // Directly modifies internal state!  
    cout << "Hacked value to: " << newValue << endl;  
}
```

Classroom Activity

```
int main() {  
    Calculator calc; // Single god object  
  
    cout << "=== TERRIBLE CALCULATOR v1.0 ===\n";  
    cout << "Commands: <op> <num>, show <type>, hack <val>, exit\n";  
    cout << "Ops: + - * / mod pow sqrt\n";  
    cout << "Show types: screen hex binary history stats\n\n";  
  
    while (true) {  
        cout << "> ";  
        string input;  
        getline(cin, input);
```

Classroom Activity

```
if (input == "exit") {  
    calc.cleanupAndExit(); // Temporal cohesion + tight coupling to exit logic  
}  
else if (input.substr(0, 4) == "show") {  
    string type = input.substr(5);  
    calc.showOutput(type); // Logical cohesion violation  
}  
else if (input.substr(0, 4) == "hack") {  
    double val = stod(input.substr(5));  
    hackCalculatorValue(calc, val); // Content coupling demonstration  
}  
else if (input == "clear history") {  
    history.clear(); // Direct manipulation of GLOBAL state (Common Coupling)  
    cout << "History wiped!\n";  
}
```

Classroom Activity

```
else if (input == "toggle debug") {  
    debugMode = !debugMode; // Global flag toggled anywhere (Common  
Coupling)  
    cout << "Debug mode: " << (debugMode ? "ON" : "OFF") << endl;  
}  
else {  
    calc.processUserInput(input); // Procedural cohesion + mixed responsibilities  
}  
}  
  
return 0;  
}
```

Classroom Activity

- ❖ Identify all types of Cohesion and Coupling violations in the code. Note down the information in this table format

Type (Coupling/Cohesion)	Location in Code	Why It's Bad
--------------------------	------------------	--------------

- ❖ Which method has the WORST cohesion? Justify your answer.
- ❖ Find 3 examples where changing one module would break another unexpectedly.
- ❖ How would you refactor this into HIGH cohesion + LOOSE coupling?
- ❖ Why is 'hackCalculatorValue()' dangerous in real systems?

Activity Solution - Cohesion

Type	Location in Code	Why It's Bad
Coincidental	Entire <code>Calculator</code> class	Mixes calculation, UI, file I/O, analytics, easter eggs
Logical	<code>showOutput()</code> method	Groups "output types" (hex/binary/history) not by purpose
Temporal	Constructor + <code>cleanupAndExit()</code>	Groups startup/shutdown tasks regardless of responsibility
Procedural	<code>processUserInput()</code>	Sequential steps (parse → validate → execute → UI → easter egg)
Communicational	<code>_updateHistory()</code>	Operates on shared <code>history</code> data (only "good" cohesion here)
Sequential	<code>performOperation()</code> → <code>saveResultToFile()</code>	Output of calc flows to file save (but poorly implemented)
Functional	Missing!	No method does only one thing well

Activity Solution - Coupling

Type	Location in Code	Why It's Bad
Content	<code>hackCalculatorValue()</code> + <code>getInternalValuePtr()</code>	External code modifies private state directly
Common	<code>history</code> , <code>currentUser</code> , <code>debugMode</code> , <code>logFile</code> globals	Any module can corrupt shared state unpredictably
External	Hardcoded <code>"results.csv"</code> , <code>system("clear")</code> , log formats	Breaks if file format/OS changes
Control	<code>performOperation(..., bool shouldLog,</code> <code>bool validateFirst)</code>	Caller controls internal behavior via flags
Stamp	<code>saveResultToFile(Calculator* calc,</code> <code>double result)</code>	Passes entire object when only <code>result</code> needed
Data	Rare! (e.g., <code>performOperation("+", 5.0)</code>)	Minimal primitive passing (the <i>only</i> good coupling here)
Message	Missing!	No event-driven/async communication

Activity Solution - Coupling

- ❖ Worst Cohesion - `processUserInput()` (procedural + mixed responsibilities)
- ❖ Global history modification, hardcoded CSV format, OS-specific `system()` calls
- ❖ Separate: `CalculatorEngine`, `HistoryManager`, `DisplayFormatter`, `FileLogger` with interfaces
- ❖ Breaks encapsulation → impossible to enforce invariants/validation

5. Software Architecture

- ❖ It refers to the **high-level structural framework of a software system**.
- ❖ It defines the **overall organization and interaction** of components, modules, and subsystems within the system.
- ❖ It establishes the foundation upon which detailed designs and implementations are built.

5. Software Architecture

Three Elements of Architecture

Components

Modular building blocks that perform specific functions within the system.

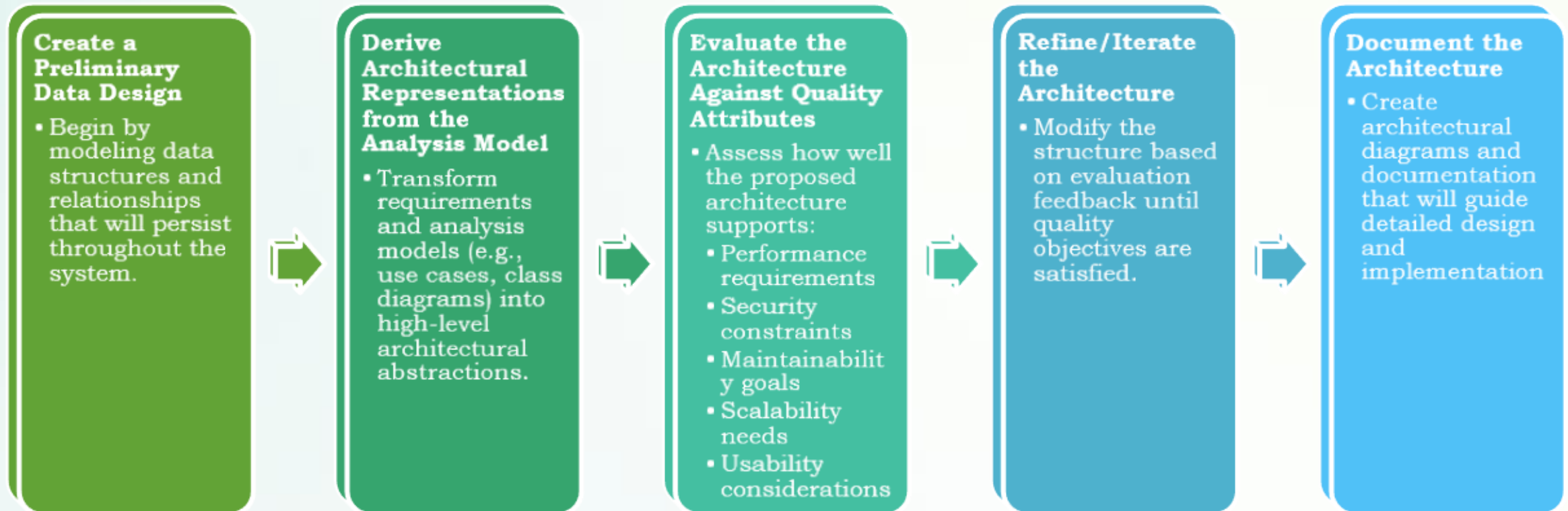
Connectors

Mechanisms that enable "communication, coordination, and cooperation" among components.

Constraints

Rules that define how components can be integrated and interact with each other.

5. Software Architecture - Architectural Design Process



5. Software Architecture

Some Architectural Patterns

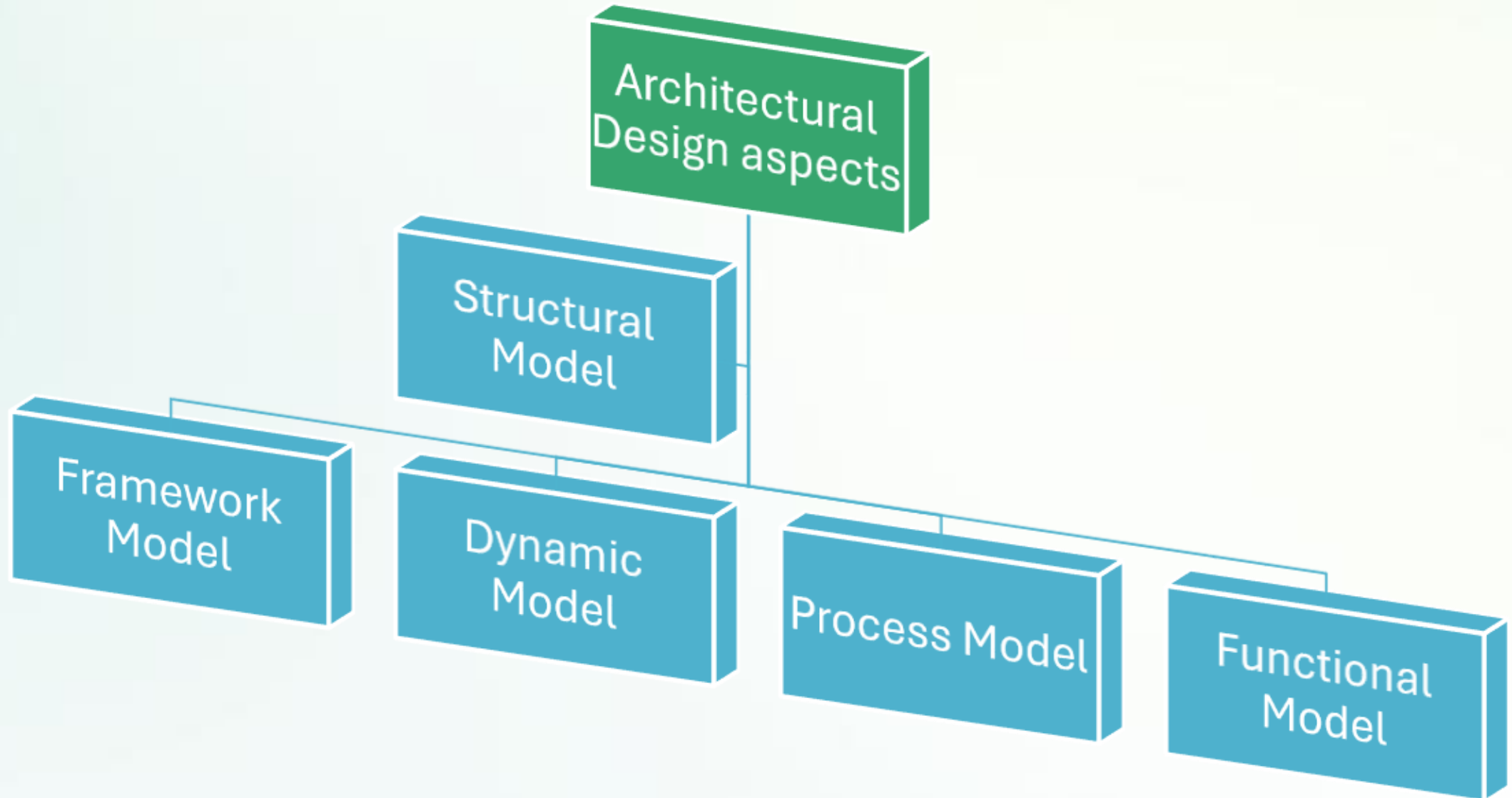
Pattern	Description	Best For...
Data-Centered	Repository-based architecture where components access a shared data store	Centralized data repository; components operate independently on shared data
Data Flow	Emphasizes incremental transformation of data as it flows through processing components	Components arranged in sequence; data moves through filters/pipes
Call and Return	Hierarchical structure where components invoke subcomponents and wait for results	Supports modularity and reuse; common in procedural systems
Object-Oriented	System organized as a collection of cooperating objects encapsulating data and behavior	Objects communicate via messages; supports inheritance and polymorphism

5. Software Architecture

Some Architectural Patterns

Pattern	Description	Best For...
Layered (N-Tier)	Organized into horizontal layers (Presentation, Business, Data).	Simple web apps; enterprise CRUD apps.
Microservices	Application is a collection of small, independent services.	Large-scale, complex systems that need to scale rapidly.
Event-Driven	Components react to "events" (state changes) asynchronously.	Real-time data processing, high-responsiveness systems.
Client-Server	Resources are centralized in a server and accessed by multiple clients.	Email, banking, and file-sharing apps.
Microkernel	A core system with "plug-in" modules for extra features.	IDEs (like VS Code) or operating systems.

5. Software Architecture



5. Software Architecture

- ❖ The Structural Model represents the static organization of the system.
- ❖ It defines what the "pieces" are and how they are physically or logically connected.

Focus	Key Tool	Why it matters
• Components, modules, classes, and their interfaces.	• UML Component Diagrams or Class Diagrams.	• It helps developers understand the boundaries of the system and where a specific piece of code should live.

5. Software Architecture

- ❖ The Framework Model is less about the specific code you write and more about the "house rules" or the environment the code lives in.
- ❖ It identifies repeatable patterns and large-scale infrastructures.

Focus

- Reusable designs, libraries, and the underlying platform (e.g., using React for the UI or Spring Boot for the backend).

Why it matters

- It ensures consistency across the project so that different teams don't "reinvent the wheel" for common tasks like logging or security.

5. Software Architecture

- ❖ The Dynamic Model describes how the **system behaves over time** and **how components talk to each other** while the app is running.

Focus	Key Tool	Why it matters
• Interactions, messages, state changes, and timing.	• Sequence Diagrams or State Machine Diagrams.	• It helps identify "bottlenecks" or race conditions where two parts of the system might try to do the same thing at once and crash.

5. Software Architecture

- ❖ The Process Model focuses on the **execution flow**.
- ❖ It bridges the gap between the **business requirements** and the **technical implementation** by looking at the system as a series of tasks.

Focus

- Concurrency, distribution, system throughput, and business logic steps.

Why it matters

- It's crucial for multi-threaded or distributed systems where you need to manage "who is doing what" at any given millisecond.

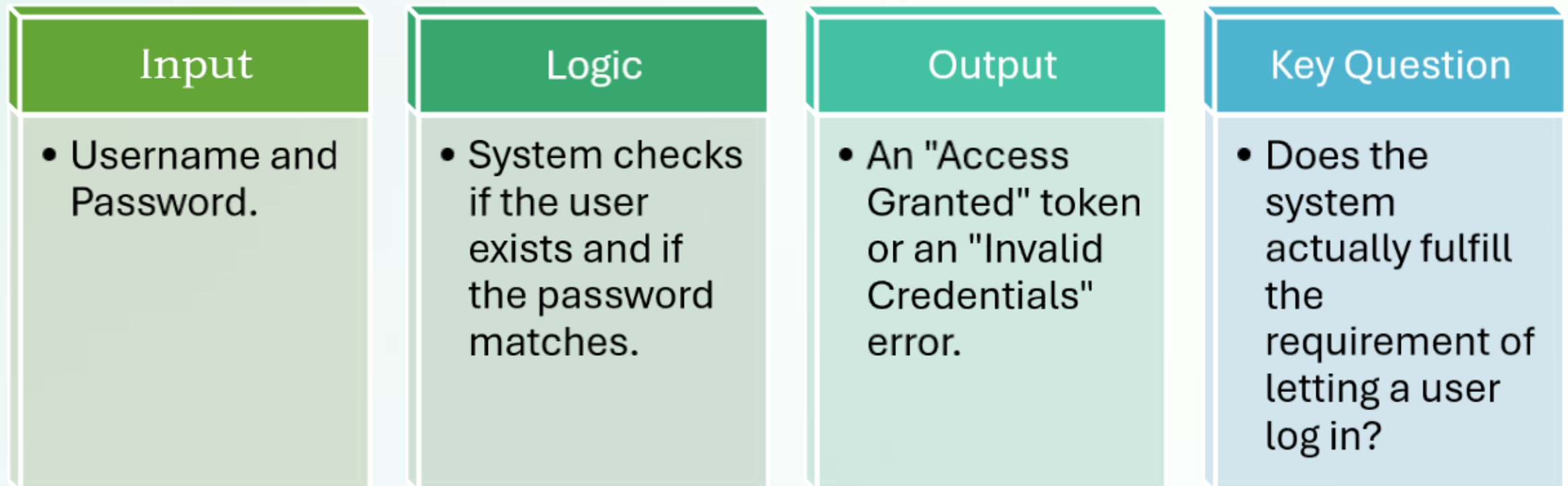
5. Software Architecture

- ❖ The Functional Model is the most high-level.
- ❖ It ignores how the system is built and focuses entirely on what it does from a user's perspective.

Focus	Key Tool	Why it matters
• Data transformations, inputs, outputs, and the mapping of user needs to system functions.	• Data Flow Diagrams (DFD) or Use Case Diagrams.	• It ensures that the architecture actually solves the problem the client paid for.

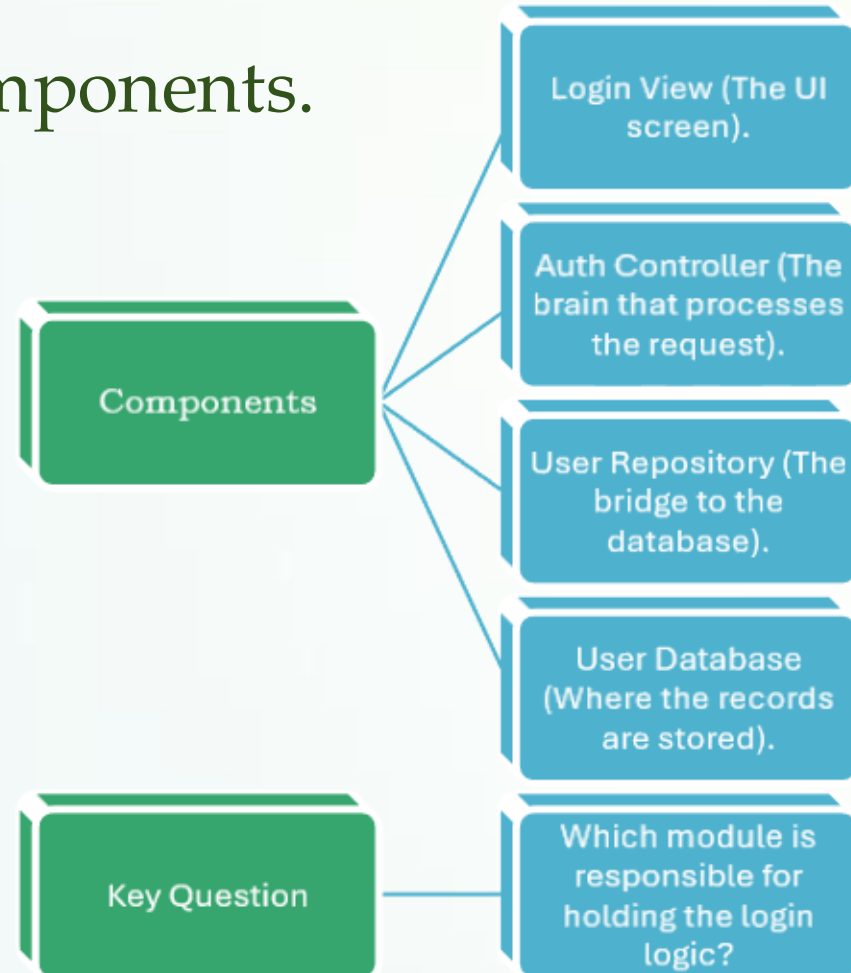
5. Software Architecture – User Login

❖ Functional Model - This model ignores the code and **focuses on the input/output and the goal.**



5. Software Architecture – User Login

❖ Structural Model - shows where the code lives. It identifies the physical or logical components.



5. Software Architecture – User Login

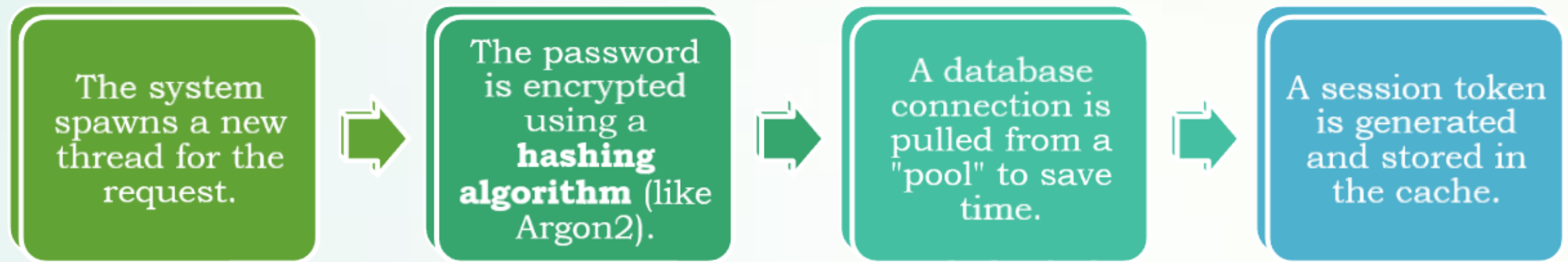
❖ Dynamic Model - tracks the sequence of messages between those components in real-time.



Key Question: *In what order do these components talk to each other to prevent errors?*

5. Software Architecture – User Login

❖ Process Model looks at the **execution**. It cares about performance, security, and "behind the scenes" work.



❖ **Key Question:** *How do we handle 1,000 people logging in at the exact same second without the server crashing?*

5. Software Architecture – User Login

❖ Framework Model - identifies the pre-made structures and standards we are following.

❖ The Rules

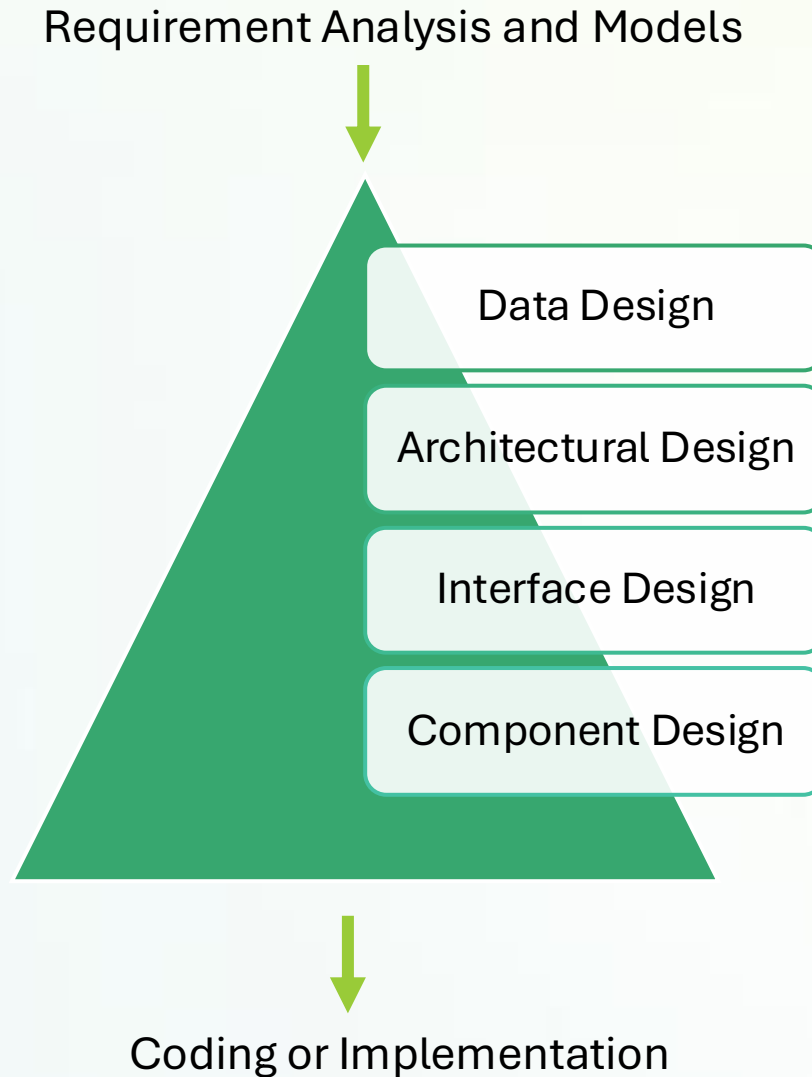
❖ We use OAuth 2.0 as the security framework.

❖ We use JWT (JSON Web Tokens) for session management.

❖ We use Spring Security (a Java framework) to handle the heavy lifting of the authentication filters.

❖ Key Question: What industry standards or third-party tools are we using to make this secure and maintainable?

Software Design Hierarchy

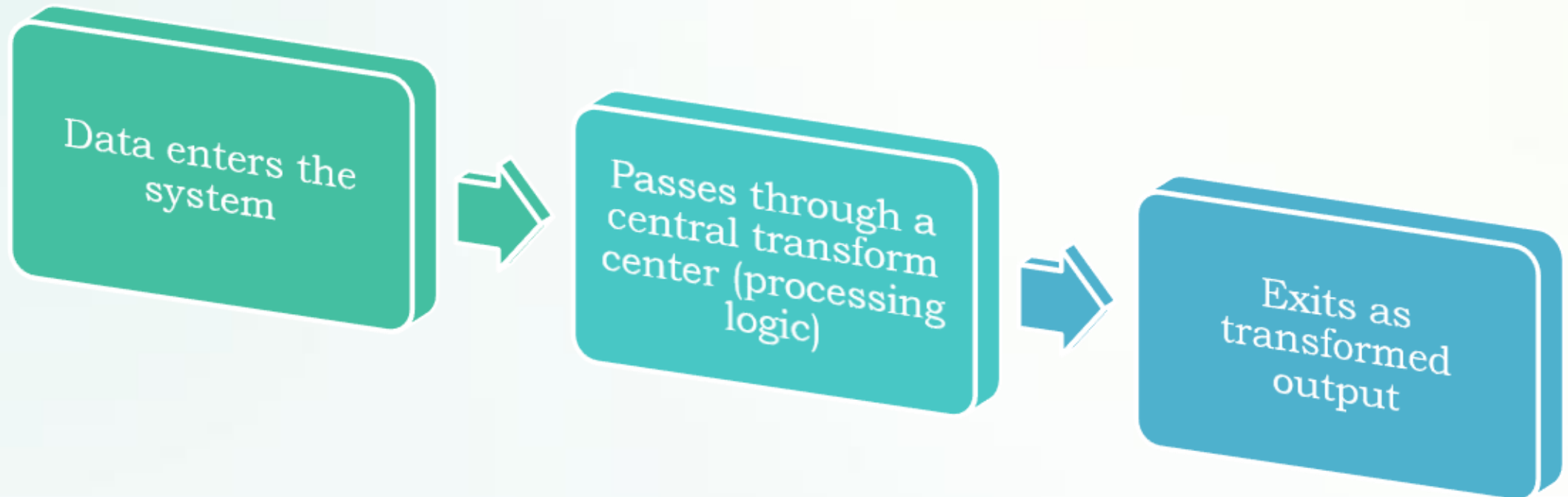


Software Design Hierarchy

Data Design	Transformation of analysis data models into data structures appropriate for implementation
Architecture Design	Definition of major structural elements (components, connectors, constraints)
Interface Design	Specification of internal/external communication mechanisms
Component-Level (Procedural) Design	Detailed algorithms and logic within components

Design - Transform

- ❖ The DFD must be analyzed to identify its dominant flow pattern.
- ❖ Systems with sequential processing: input → transformation → output (e.g., compilers, signal processors)



Design - Transform

❖ **Transform mapping** converts a DFD with transform flow characteristics into a **modular program structure** (typically represented as a structure chart).

Design – Transform Mapping Steps

❖ Review the Fundamental System Model

- ❖ Examine the context-level DFD to understand system boundaries and external entities.

❖ Refine the DFD

- ❖ Ensure the DFD is complete, consistent, and exhibits clear transform flow characteristics.

❖ Identify Transform Center Boundaries

- ❖ Locate the transform center: the core processing logic where input data is converted to output
- ❖ Define incoming flow boundary: where external input data transitions to internal data format
- ❖ Define outgoing flow boundary: where internal data transitions to external output format

Design - Transform

❖ First-Level Factoring

- ❖ Create a top-level control module that:
 - ❖ Calls an input module (handles incoming flow)
 - ❖ Calls a transform module (executes the transform center)
 - ❖ Calls an output module (handles outgoing flow)

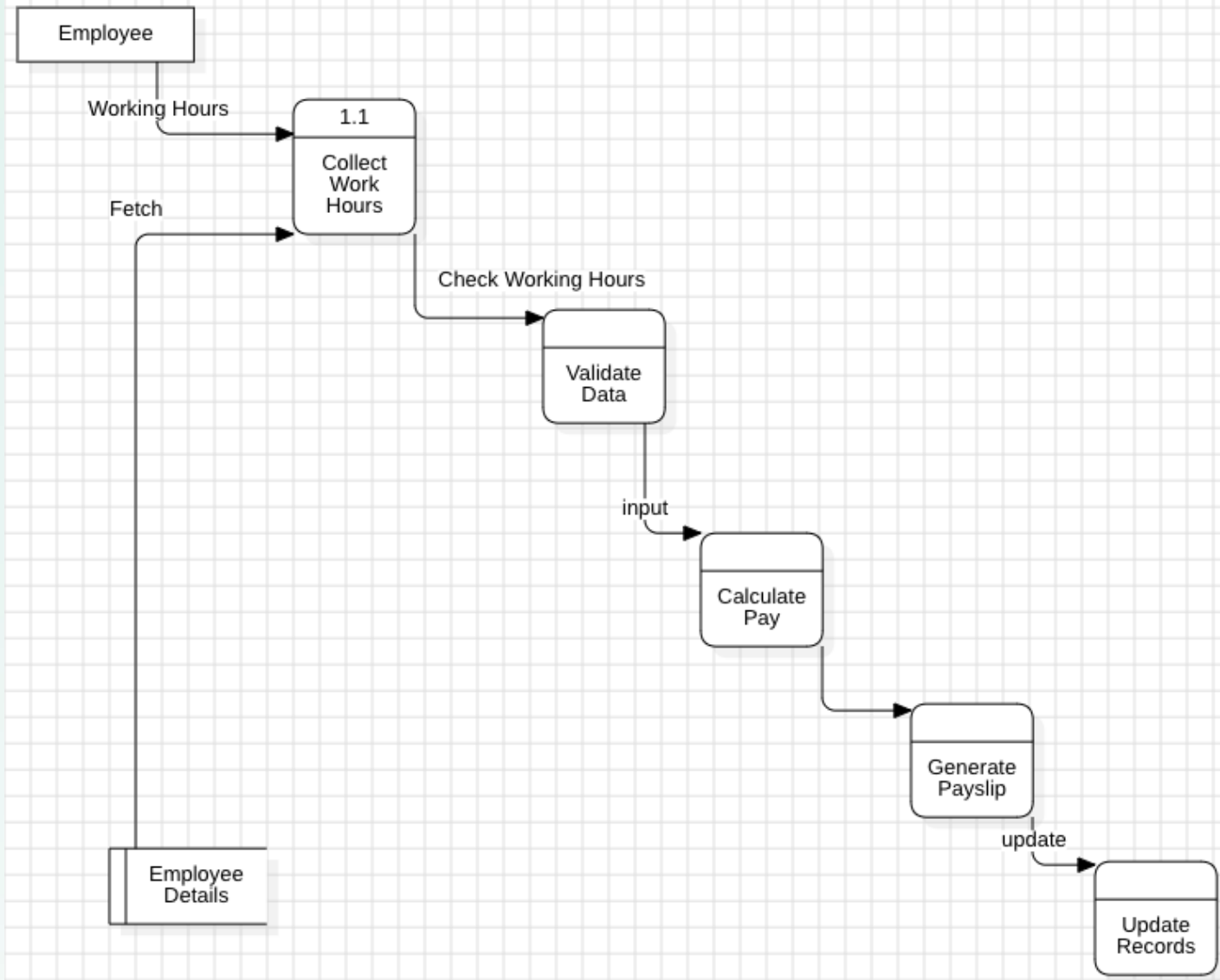
❖ Second-Level Factoring

- ❖ Decompose each first-level module into subordinate modules representing individual processes/bubbles from the DFD.

❖ Refine the Structure Chart

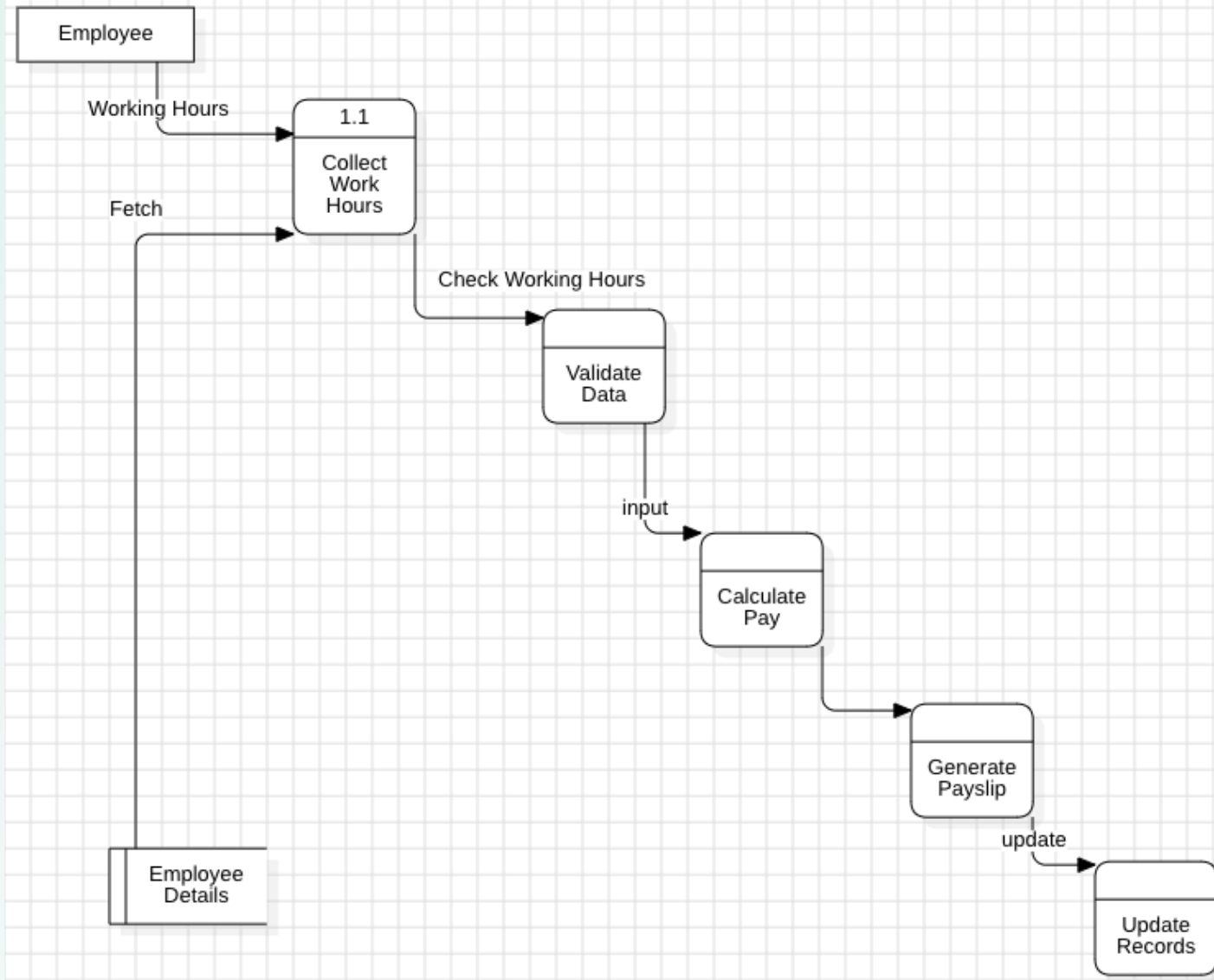
- ❖ Apply design heuristics (e.g., reduce fan-out, improve cohesion, minimize coupling) to optimize modularity

Design – Transform - Example



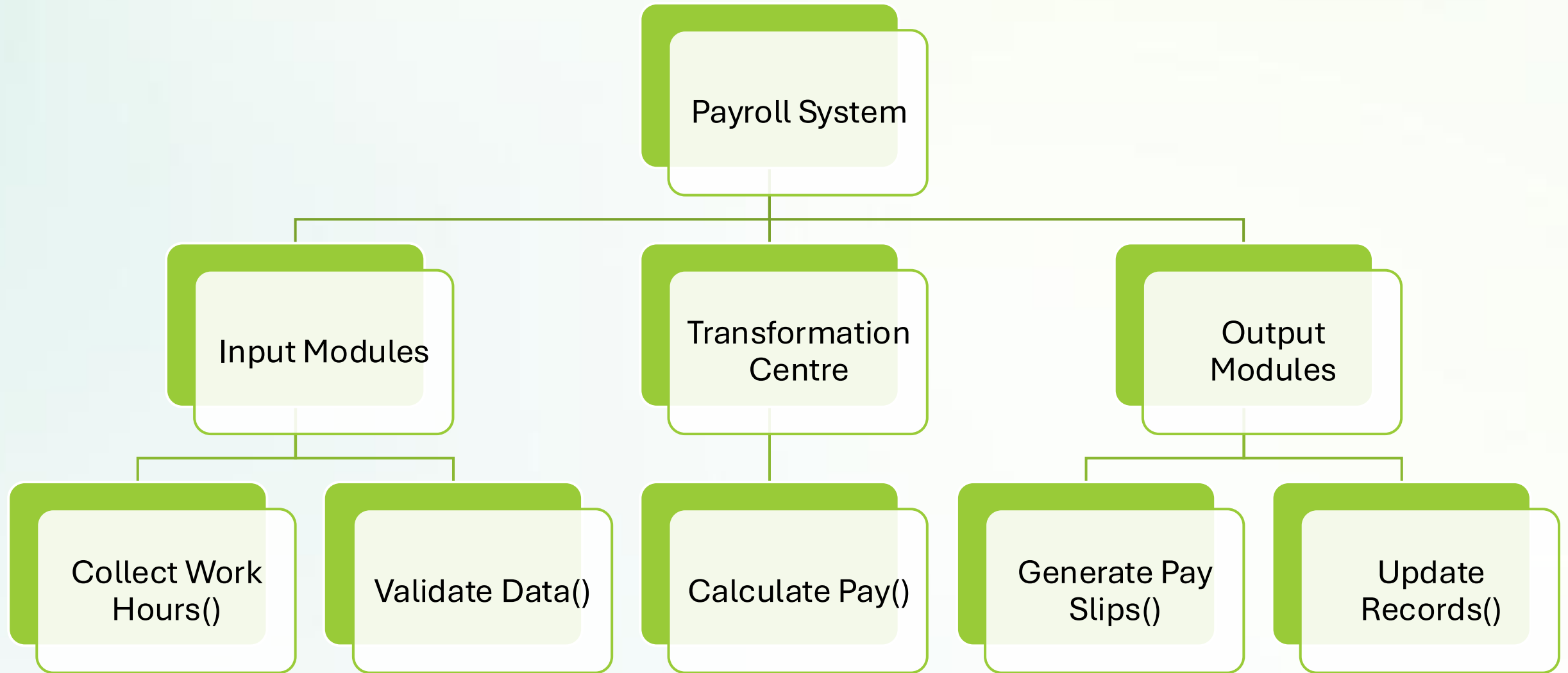
DFD is given. Perform Transform Mapping

Design – Transform - Example



Input domain	Transform Domain	Output Domain
Collect Work Hours()	Calculate Pay()	Generate Pay Slips()
Validate Data()		Update Records()

Design – Transform - Example



Design – Transaction

❖ Transaction mapping addresses DFDs where a single transaction triggers one of multiple action paths

❖ Identify the Transaction Center

- ❖ Locate the process that:
 - ❖ Receives the transaction input
 - ❖ Analyzes the transaction type
 - ❖ Dispatches control to the appropriate action path

❖ Isolate Incoming Flow Path

- ❖ Identify the path that delivers the transaction to the transaction center.

❖ Identify and Factor Action Paths

- ❖ For each possible transaction type:
 - ❖ Determine its flow characteristics (transform or transaction)
 - ❖ Apply appropriate mapping technique to each path

Design – Transaction

❖ First-Level Factoring

- ❖ Create a top-level transaction control module that:
 - ❖ Receives the transaction
 - ❖ Determines transaction type
 - ❖ Dispatches to the appropriate action-path module

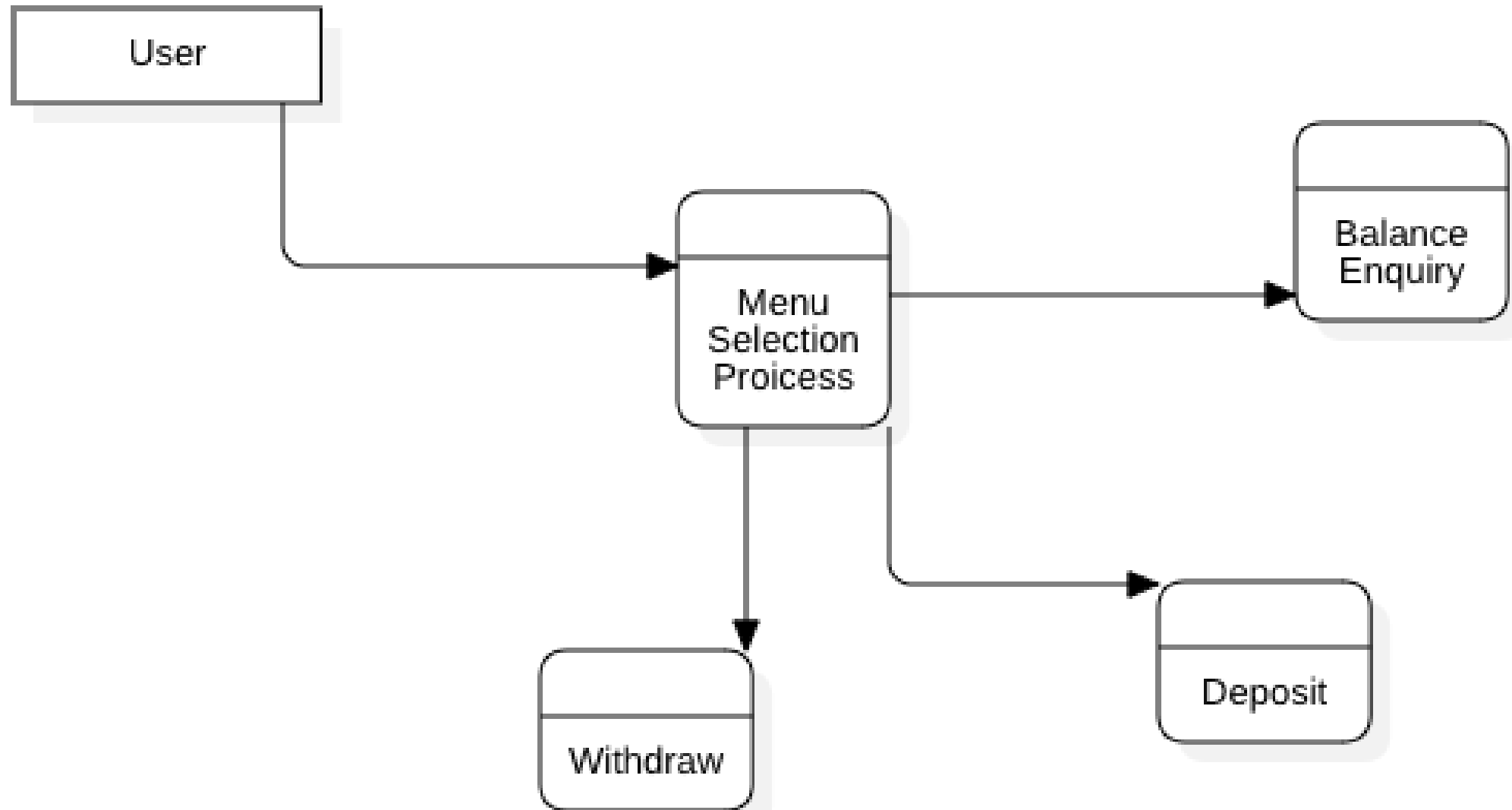
❖ Second-Level Factoring

- ❖ Decompose each action-path module using transform or transaction mapping as appropriate.

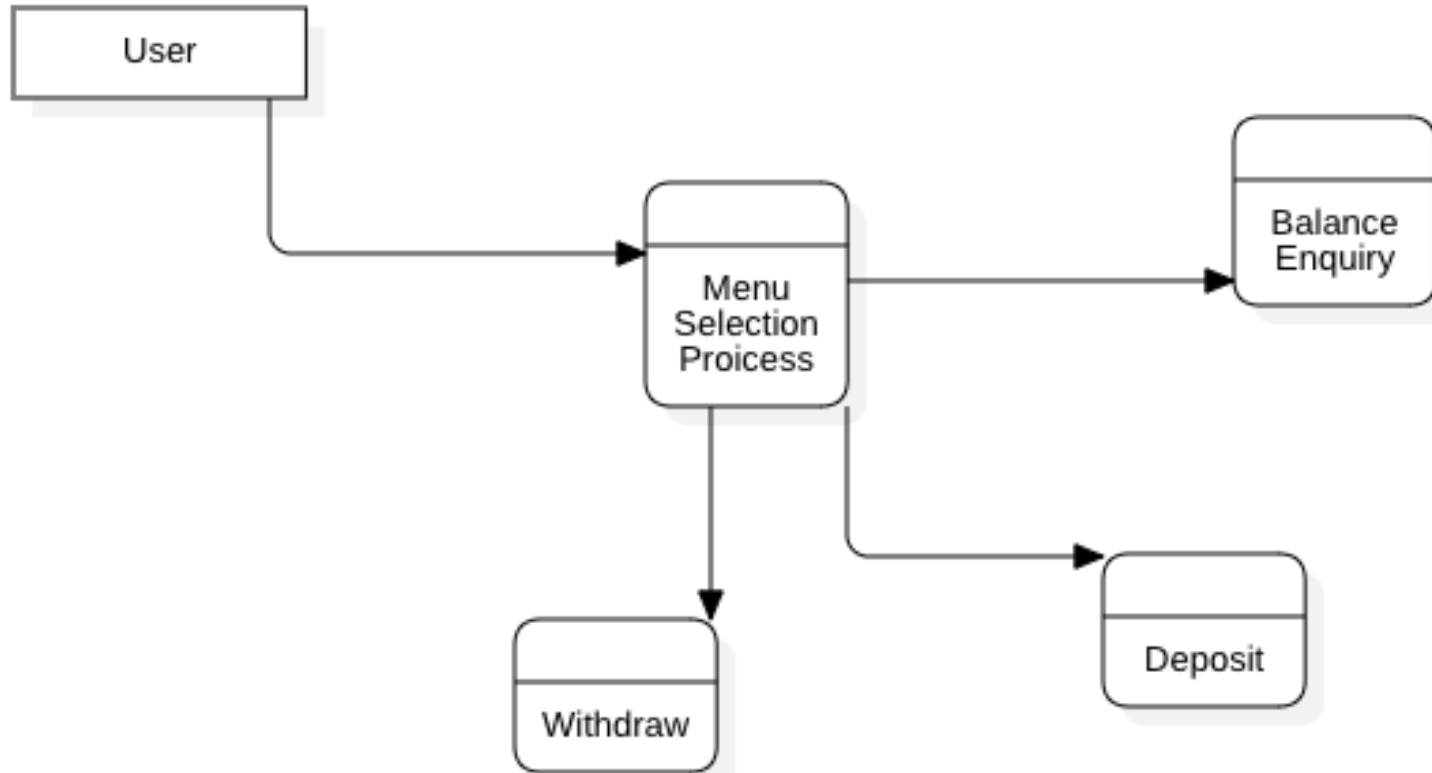
❖ Refine the Structure

- ❖ Apply modularity heuristics to improve the design quality.

Design – Transaction - Example

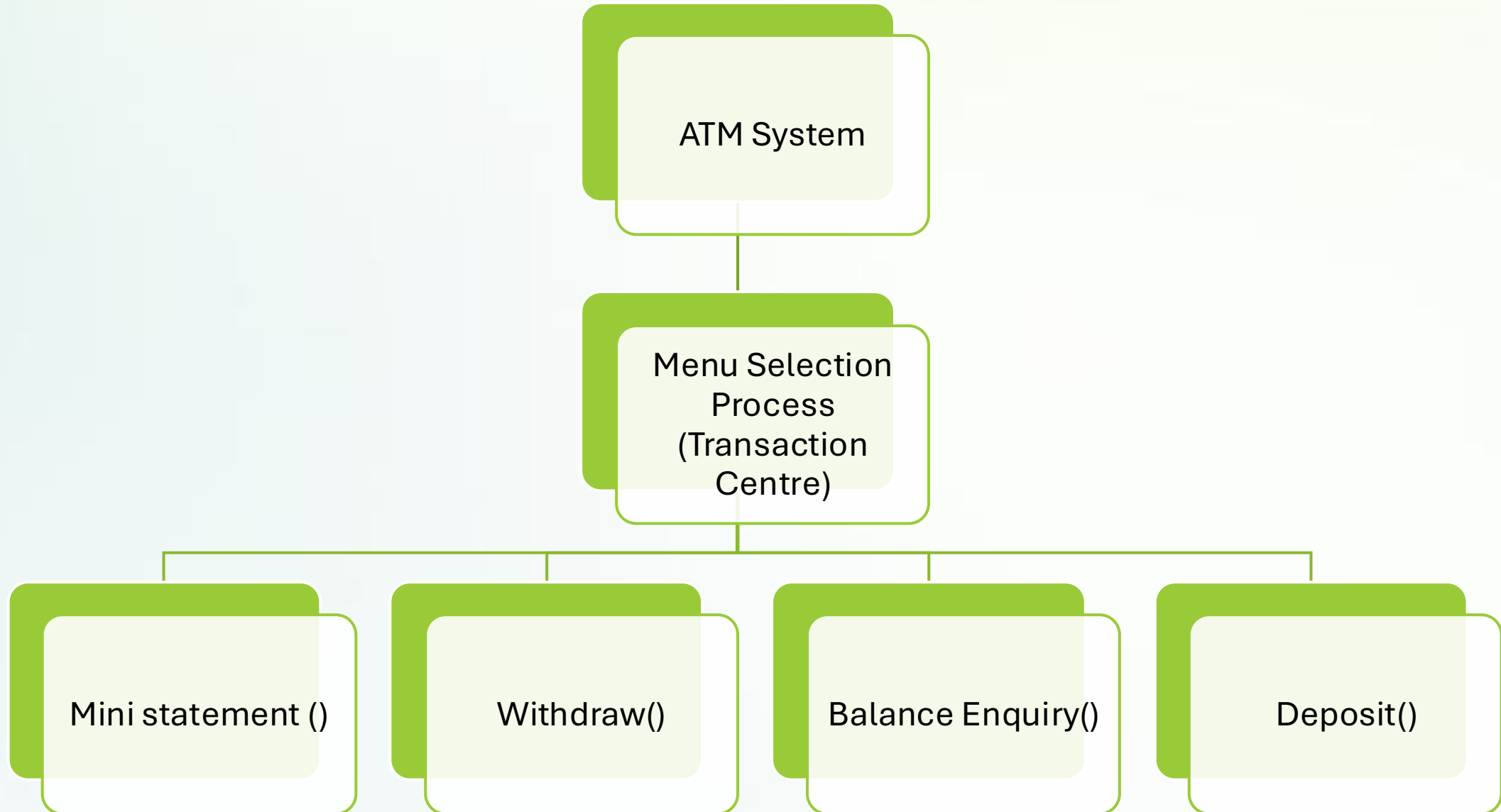


Design – Transaction - Example

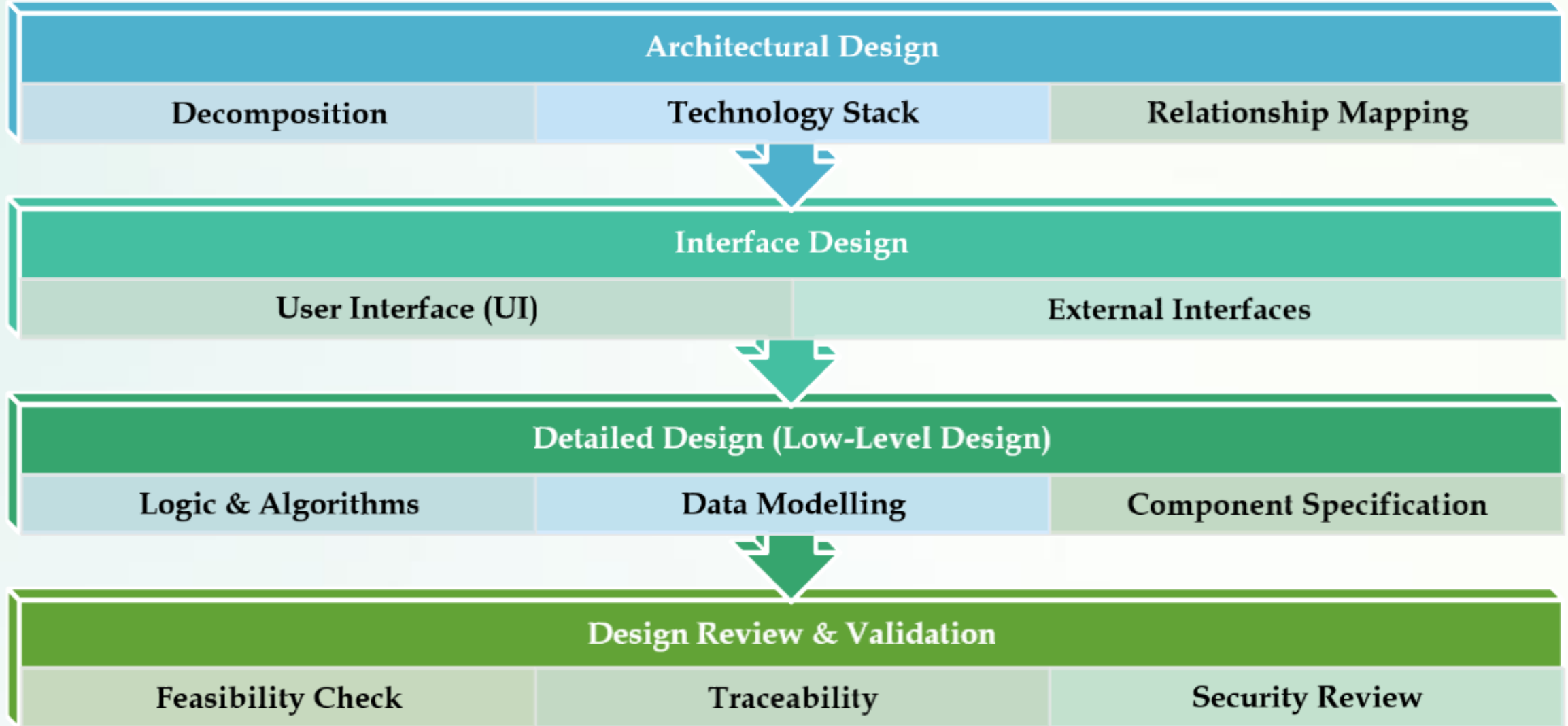


- ❖ Transaction Center: "Menu Selection Process"
- ❖ The system branches into different transactions (Withdraw, Deposit, Balance Inquiry) based on user selection.

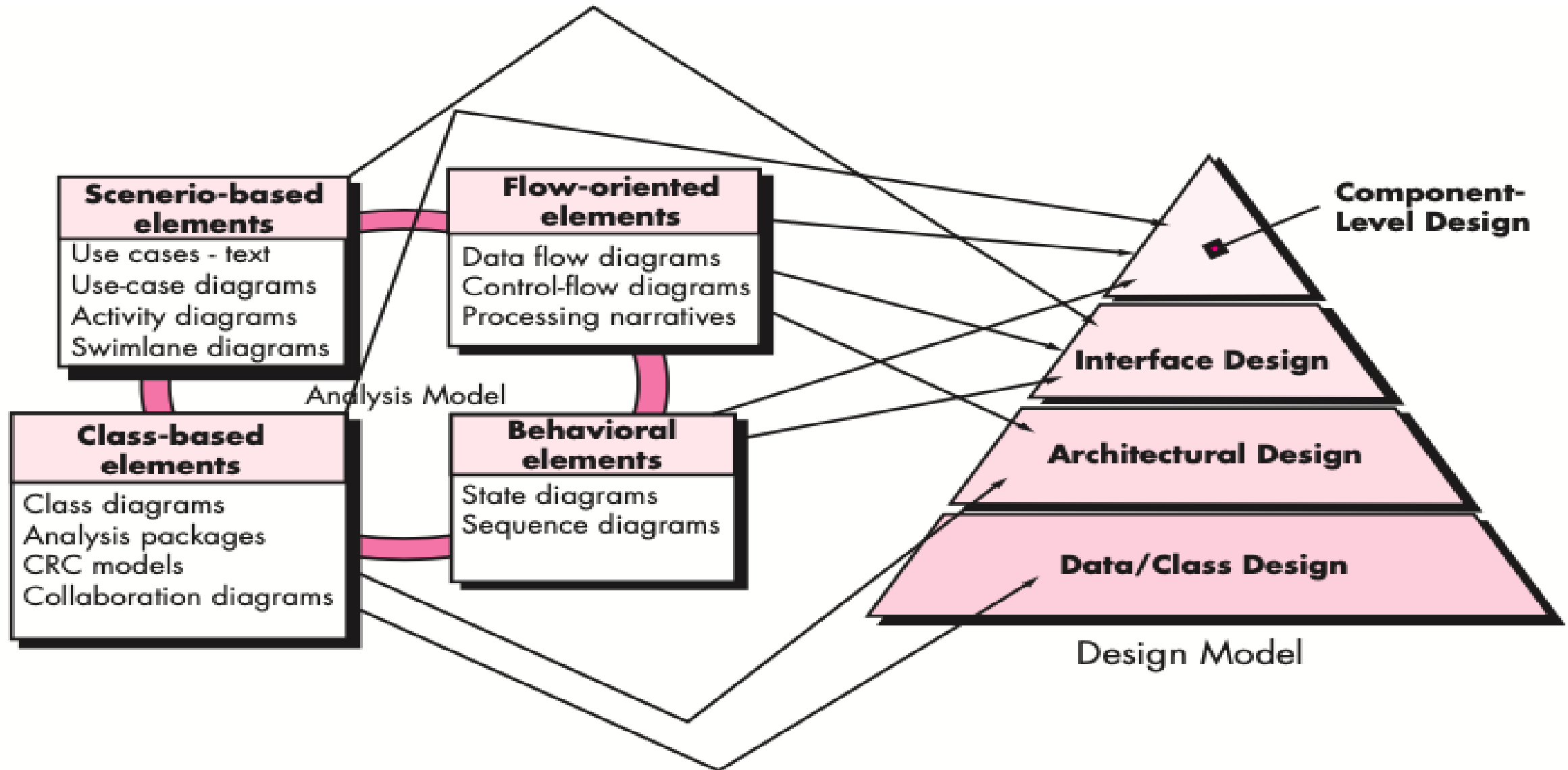
Design – Transaction - Example



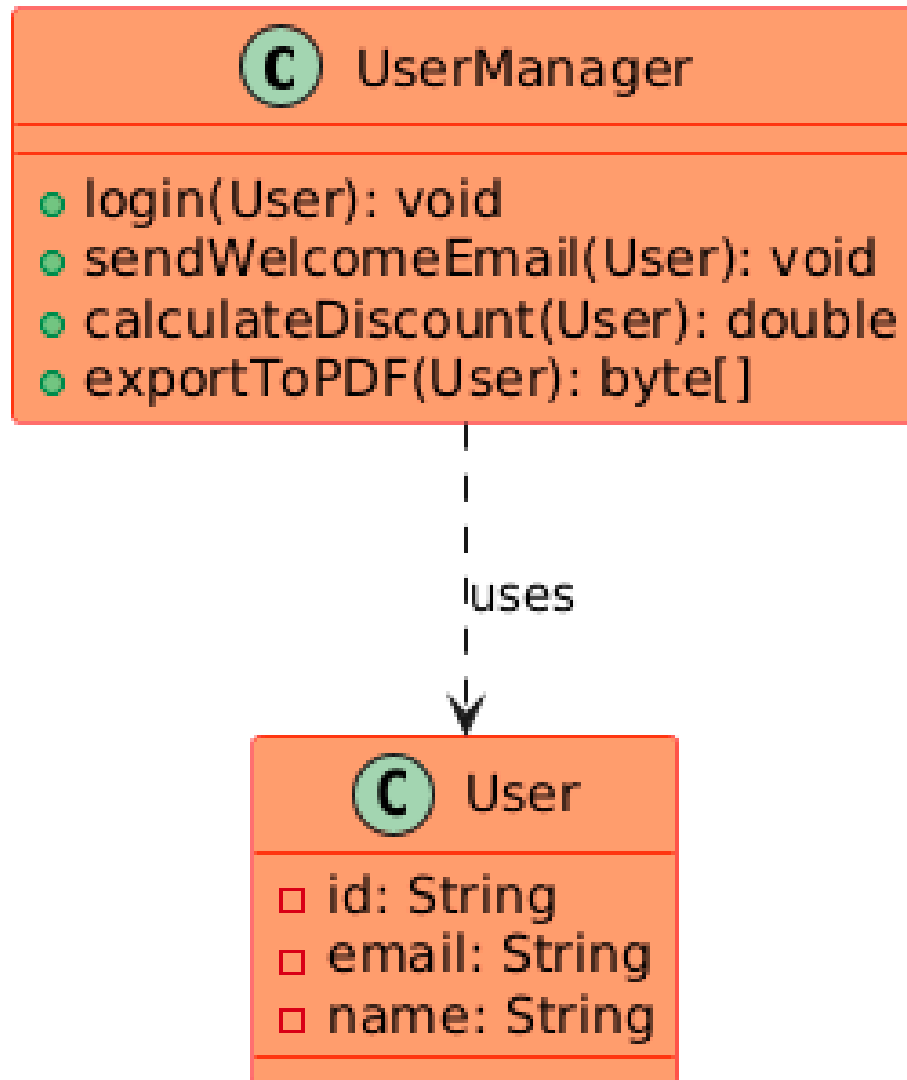
Overall Design Process



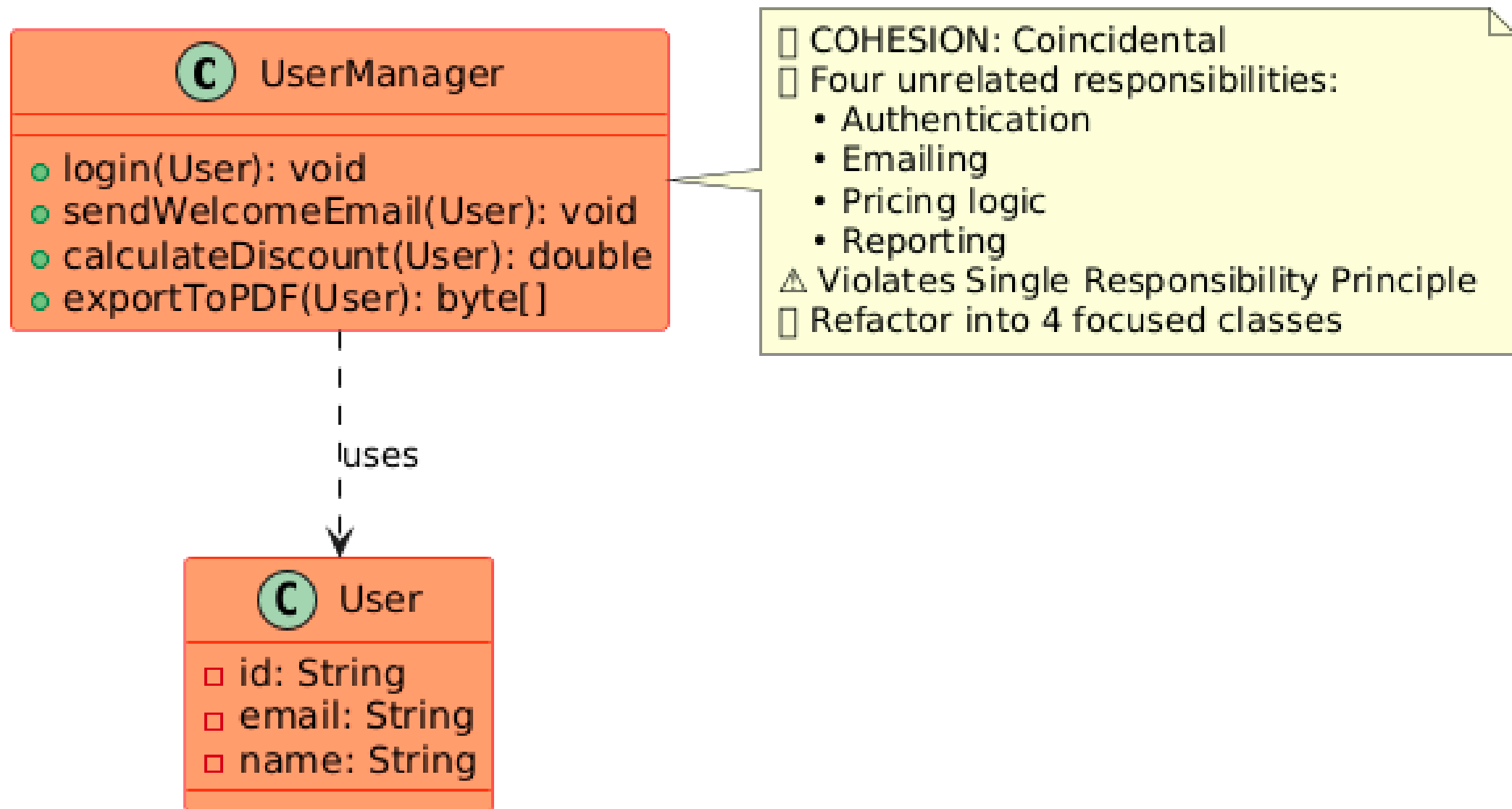
Translating Requirement Model into Design Model



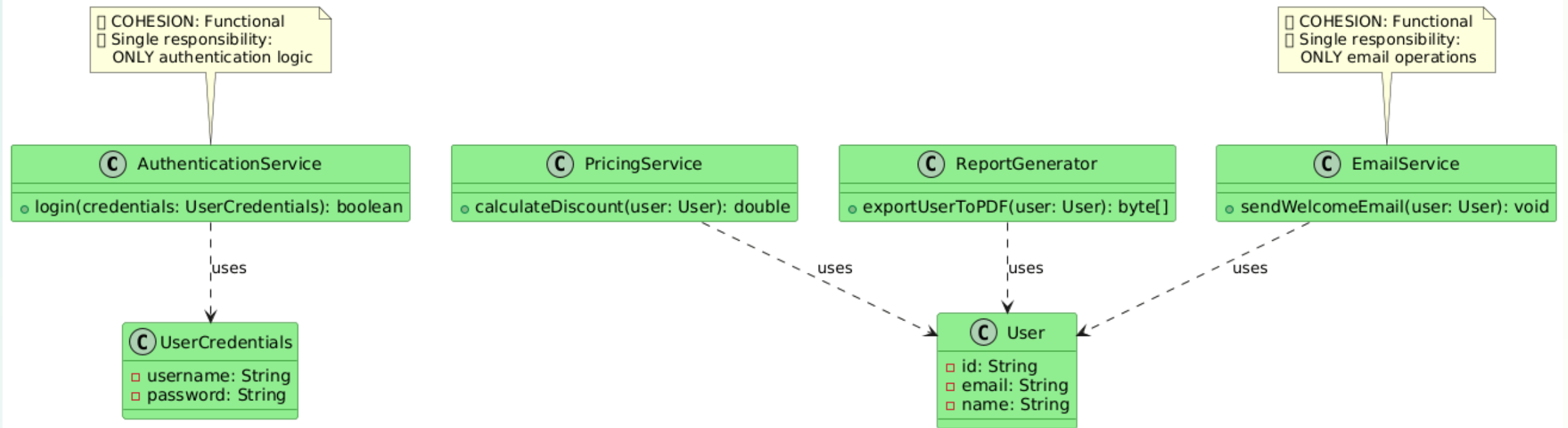
Refactoring of Designs



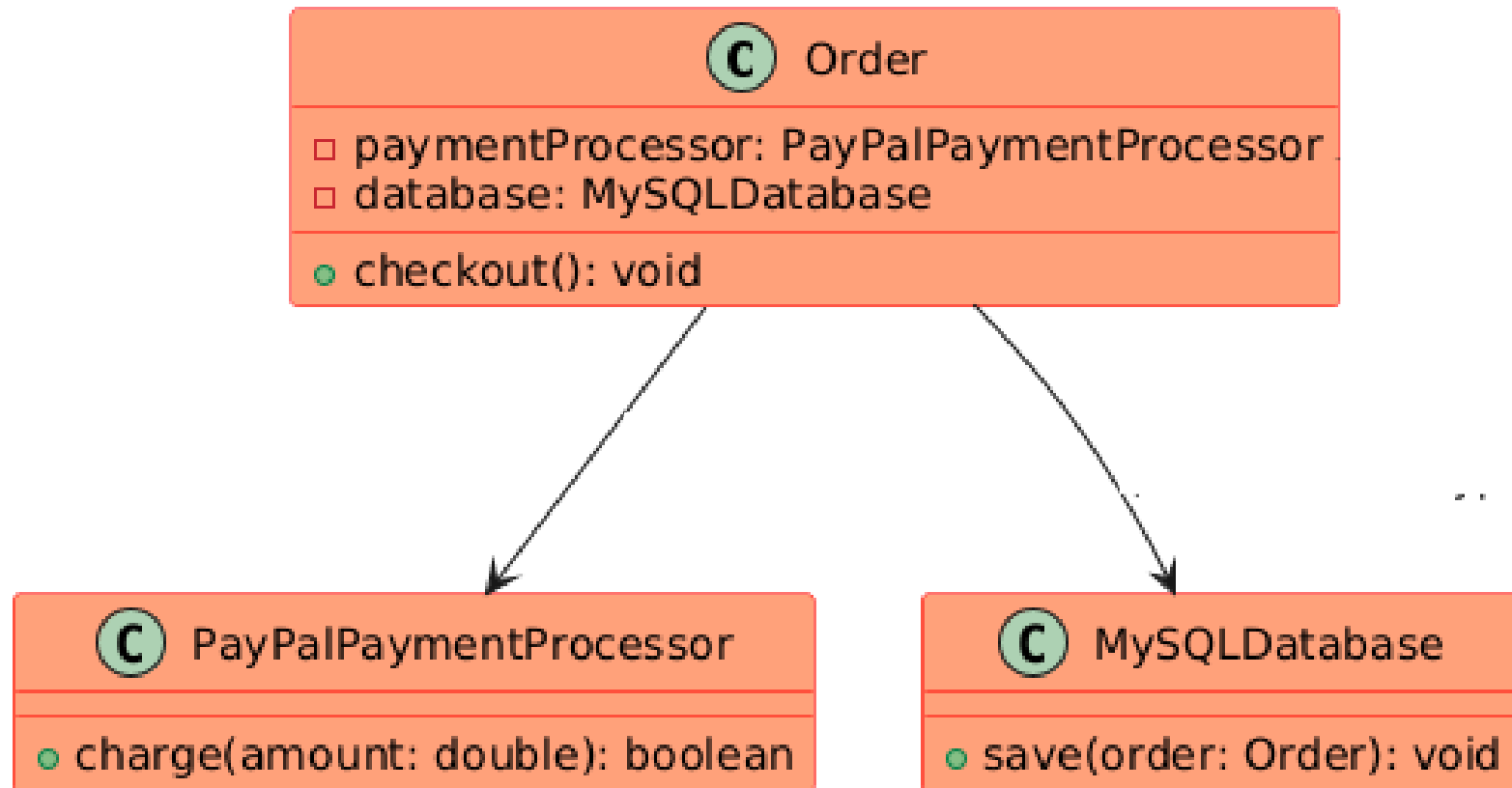
Refactoring of Designs



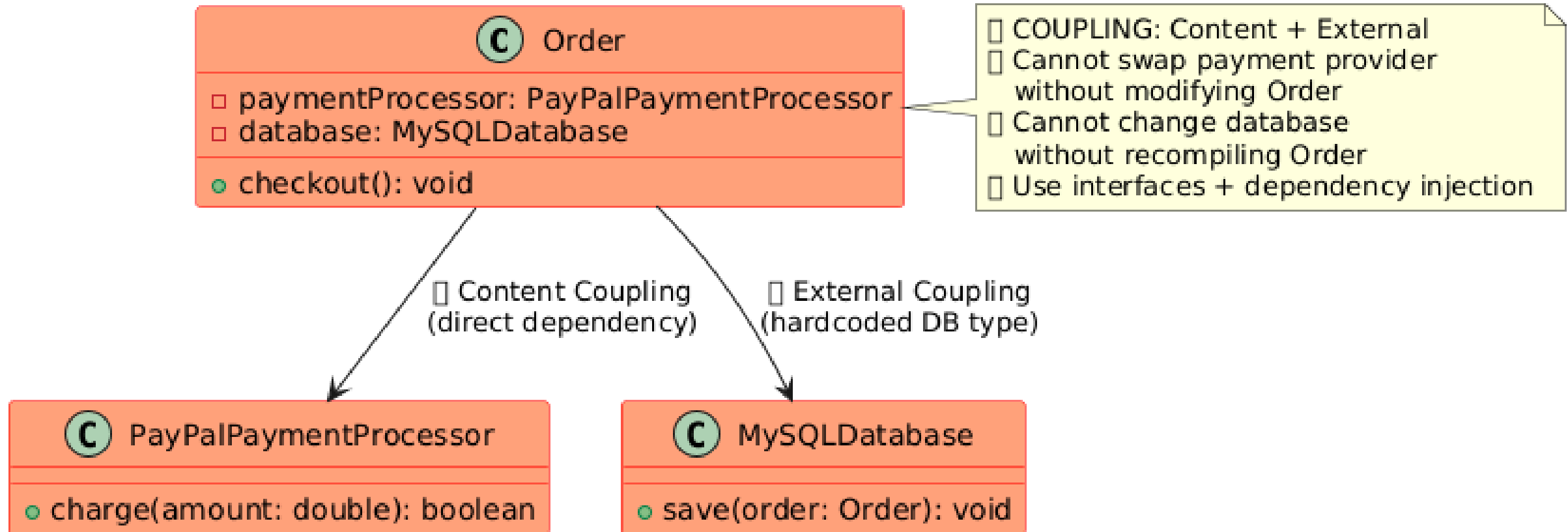
Refactoring of Designs



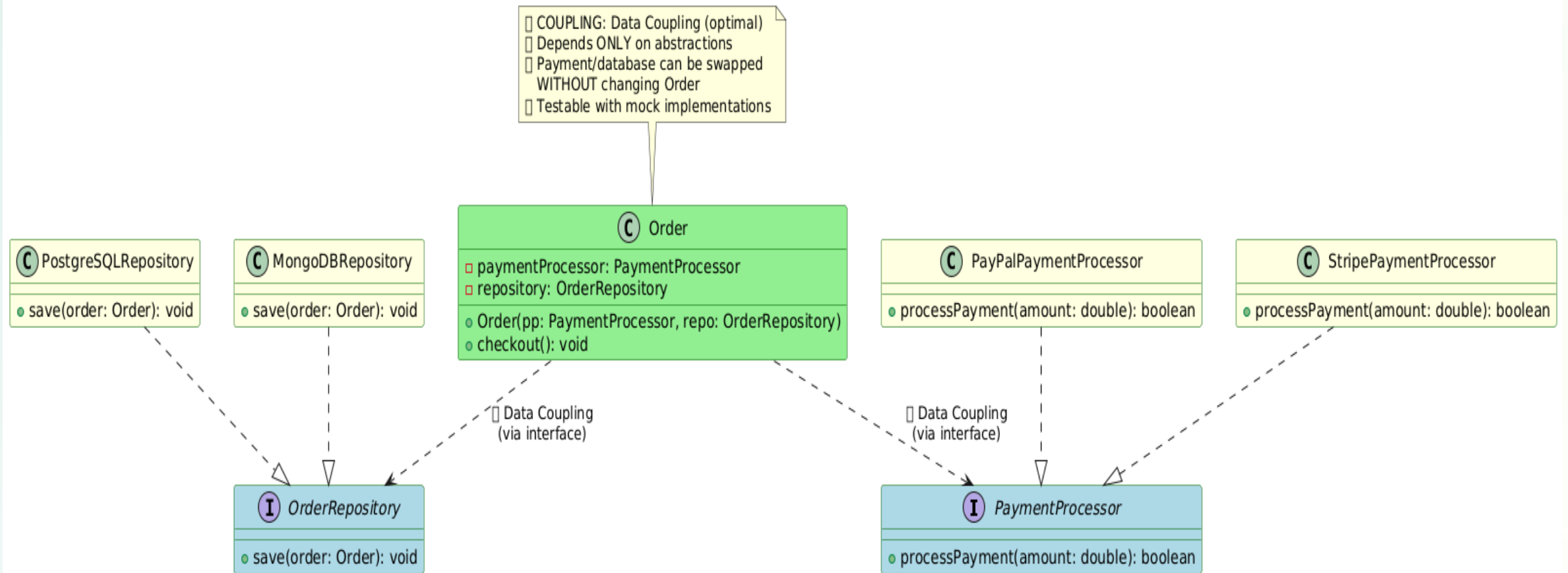
Refactoring of Designs



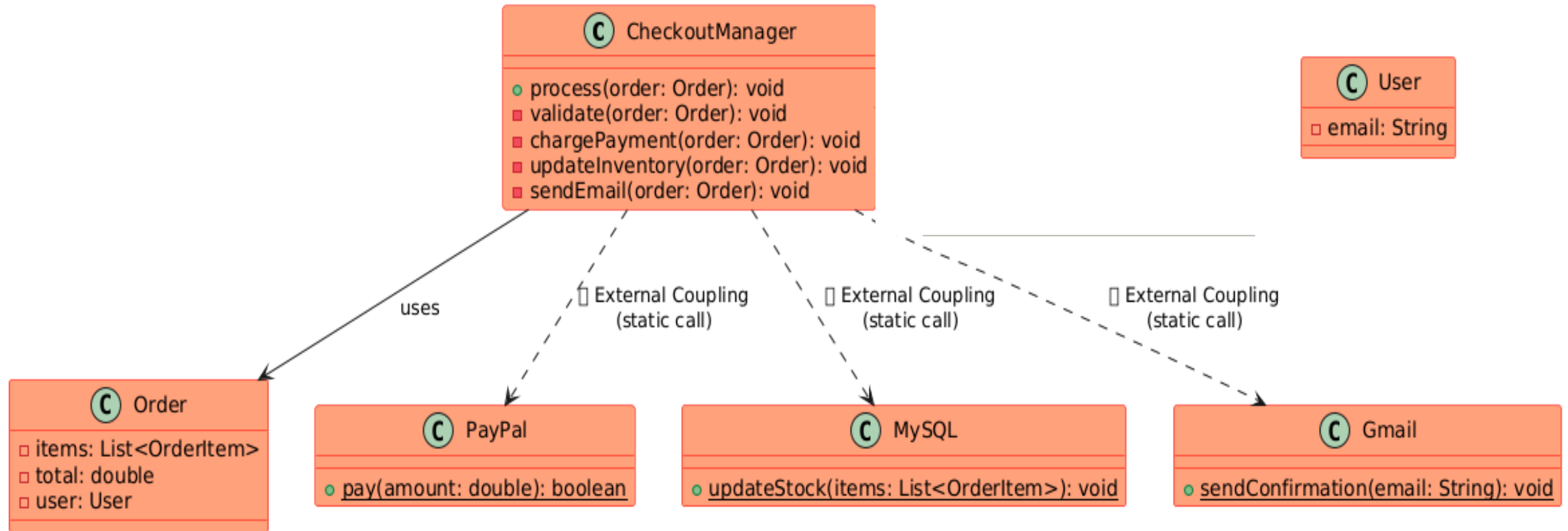
Refactoring of Designs



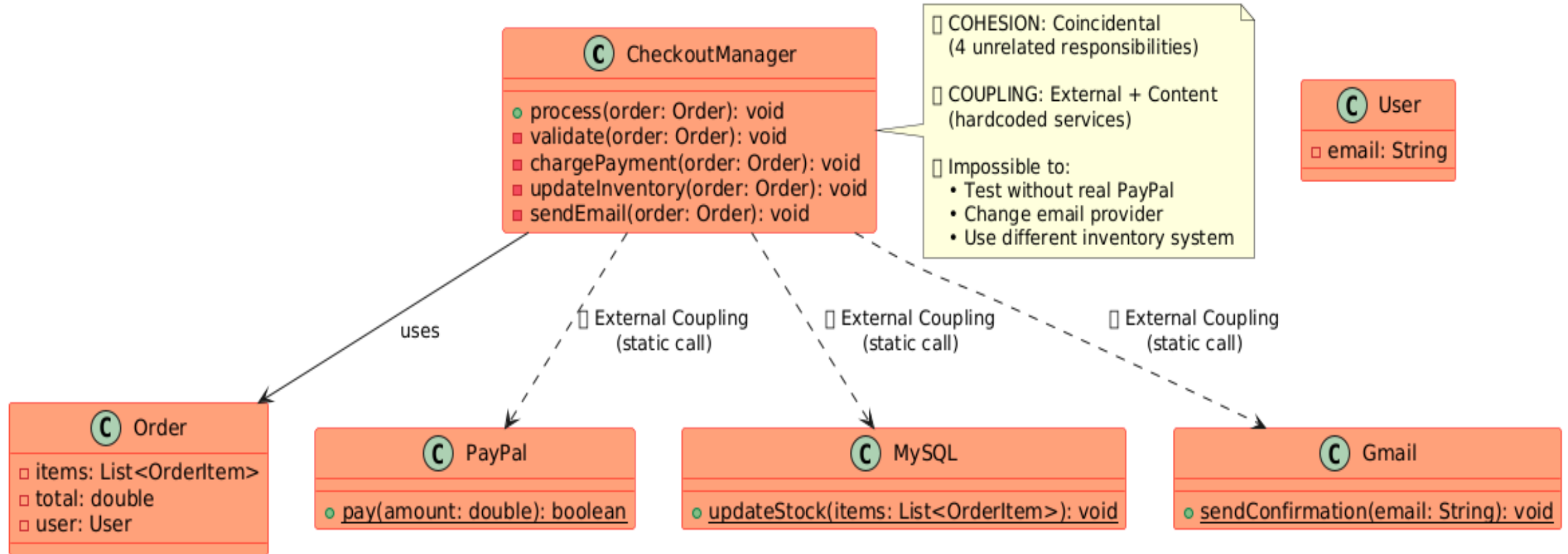
Refactoring of Designs



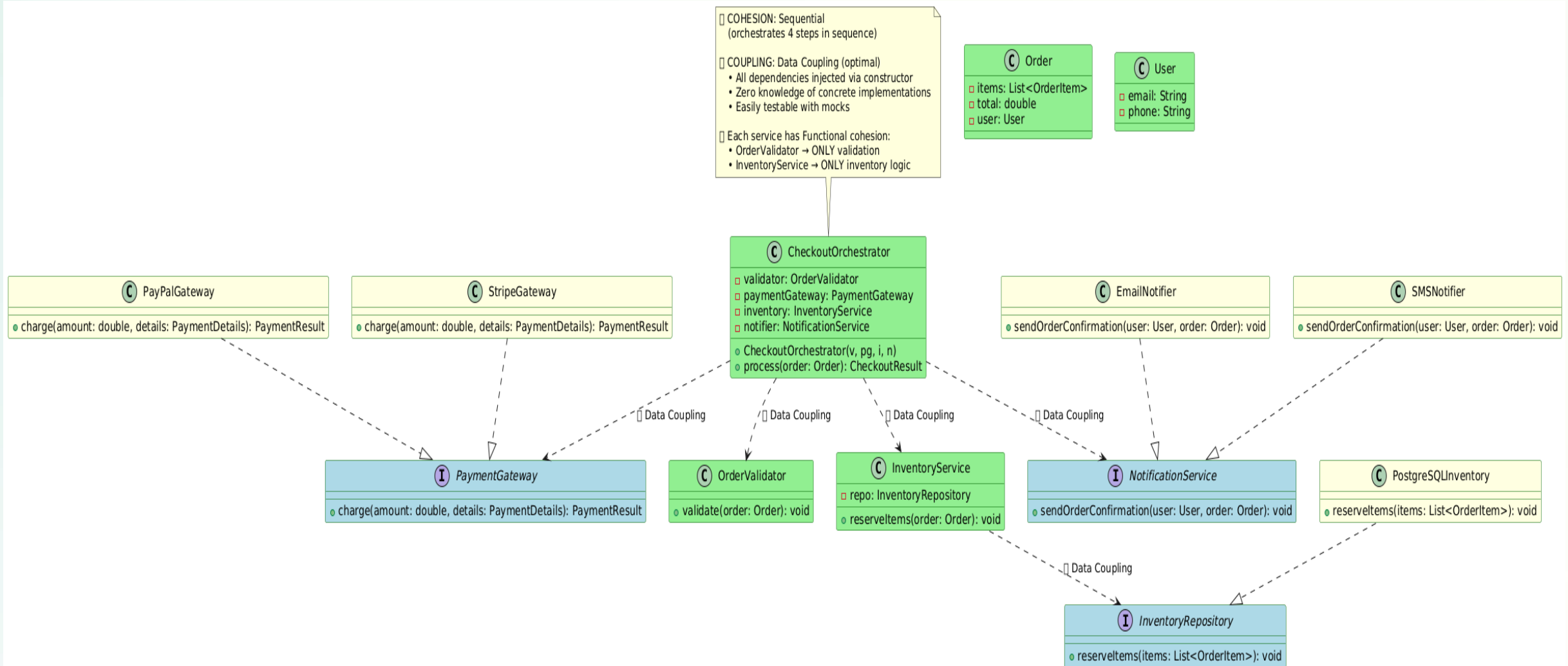
Refactoring of Designs



Refactoring of Designs



Refactoring of Designs



User Interface Design

- ❖ User Interface (UI) Design refers to the **process of designing** the **visual and interactive elements** of a digital product, such as websites, mobile apps, or software applications.
- ❖ The goal of UI design is to **create an interface** that is **visually appealing, easy to use, and intuitive** for users to interact with.
- ❖ It focuses on **how users engage** with the product, ensuring that the design **enhances usability** and provides a **seamless experience**.

User Interface Design



- It is about the look and feel – the aesthetics, layout, typography, colors, and visual elements.



- It is about the overall experience – how the user feels while interacting with the product, including usability, accessibility, and functionality.

User Interface Design – Key Elements

❖ Visual Design

- ❖ **Typography:** Choosing fonts that are readable and appropriate for the brand or purpose.
- ❖ **Color Palette:** Using colors strategically to guide attention, convey meaning, and maintain consistency.
- ❖ **Imagery:** Incorporating icons, illustrations, and images that align with the product's tone and style.

❖ Layout and Structure

- ❖ **Grid Systems:** Organizing content in a structured and balanced way using grids.
- ❖ **Spacing:** Ensuring proper spacing between elements to avoid clutter and improve readability.
- ❖ **Hierarchy:** Prioritizing important elements through size, color, or placement.

User Interface Design – Key Elements

❖ Interactivity

- ❖ **Buttons and Controls:** Designing buttons, sliders, toggles, and other interactive elements to be clear and functional.
- ❖ **Animations and Transitions:** Adding subtle animations to enhance feedback and make interactions smoother.
- ❖ **Micro-interactions:** Small, engaging interactions (e.g., a button changing color when clicked) that improve the user experience.

❖ Accessibility

- ❖ Ensuring the interface is usable by people with disabilities, such as those who rely on screen readers or have color blindness.
- ❖ Following accessibility guidelines like WCAG (Web Content Accessibility Guidelines).

User Interface Design – Key Elements

❖ Consistency

- ❖ Maintaining a consistent design language across all screens and components to avoid confusion and build trust.

❖ Responsiveness

- ❖ Designing interfaces that adapt to different screen sizes and devices (desktops, tablets, smartphones).

User Interface Design – Models

User Model

- Establishes the profile of end users, including demographics, abilities, and goals.
 - Users can be categorized as:
 - **Novices** : No syntactic or semantic knowledge of the system.
 - **Knowledgeable, Intermittent Users** : Understand the application but struggle with interface details.
 - **Knowledgeable, Frequent Users** : Skilled users who seek shortcuts and efficient interactions.

Design Model

- Created by software engineers to align with the user model and guide the design process.

Mental Model (system Perception)

- The user's internal understanding of how the system works, influenced by their experience and familiarity.

Implementation Model

- Combines the system's outward appearance (look and feel) with supporting documentation (manuals, help files).
- Goal - Align the implementation model with the user's mental model for effective and comfortable use.

User Interface Design – Process

Research

Understand the target audience, their needs, and pain points.

Analyze competitors and industry trends.

Wireframing

Create low-fidelity sketches or wireframes to outline the structure and layout of the interface.

Prototyping

Build interactive prototypes to test the flow and functionality of the design.

Visual Design

Apply colors, typography, and imagery to create high-fidelity mockups.

Testing

Conduct usability testing to gather feedback from real users and identify areas for improvement.

Iteration

Refine the design based on feedback and testing results.

Implementation

Collaborate with developers to bring the design to life in the final product.

User Interface Design – Golden Rules

Rule	Why It Matters	Example
Strive for Consistency	Reduces learning curve	Same button styles across pages
Enable Frequent Users to Use Shortcuts	Increases efficiency	Keyboard shortcuts
Offer Informative Feedback	Builds trust and clarity	Loading spinners, success messages
Design Dialogs to Yield Closure	Provides a sense of completion	Progress indicators
Prevent Errors	Saves time and reduces frustration	Input validation, confirmation dialogs
Permit Easy Reversal of Actions	Encourages exploration	Undo/redo functionality
Support Internal Locus of Control	Empowers users	Customizable settings

User Interface Design – Golden Rules

Rule	Why It Matters	Example
Reduce Short-Term Memory Load	Enhances usability	On-screen hints
Keep Interfaces Simple and Natural	Improves intuitiveness	Familiar icons and metaphors
Provide Clear Navigation	Prevents confusion	Breadcrumbs, persistent menus
Be Forgiving	Encourages continued use	Editable forms, helpful error messages
Focus on Accessibility	Ensures inclusivity	High contrast, alt text
Test and Iterate	Ensures the design meets user needs	Usability testing, A/B testing